

C++ Standard Library Issues to be moved in Virtual Plenary, Nov. 2020

Doc. no. P2236R0

Date: 2020-10-14

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

- [Ready Issues](#)
- [Tentatively Ready Issues](#)
- [Tentatively NAD Issues](#)

Ready Issues

These 13 issues were moved to **Ready** at the end of the Prague meeting.

[2839](#). Self-move-assignment of library types, again

Section: 16.4.6.16 [[lib.types.movedfrom](#)], 16.4.5.9 [[res.on.arguments](#)], 22.2.1 [[container.requirements.general](#)] **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2016-12-09 **Last modified:** 2020-09-06

Priority: 2

Discussion:

LWG [2468](#)'s resolution added to `MoveAssignable` the requirement to tolerate self-move-assignment, but that does nothing for library types that aren't explicitly specified to meet `MoveAssignable` other than make those types not meet `MoveAssignable` any longer.

To realize the intent here, we need to carve out an exception to 16.4.5.9 [[res.on.arguments](#)]'s restriction for move assignment operators and specify that self-move-assignment results in valid but unspecified state unless otherwise specified. The proposed wording below adds that to 16.4.6.16 [[lib.types.movedfrom](#)] since it seems to fit well with the theme of the current paragraph in that section.

In addition, to address the issue with 22.2.1 [[container.requirements.general](#)] noted in LWG [2468](#)'s discussion, the requirement tables in that subclause will need to be edited in a way similar to LWG [2468](#).

[2017-01-27 Telecon]

Priority 2

[2018-1-26 issues processing telecon]

Status to 'Open'; Howard to reword using 'MoveAssignable'.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4618](#).

1. Add a new paragraph at the end of 16.4.6.16 [[lib.types.movedfrom](#)]:

-1- Objects of types defined in the C++ standard library may be moved from (12.8). Move operations may be explicitly specified or implicitly generated. Unless otherwise specified, such moved-from objects shall be placed in a valid but unspecified state.

-?- An object of a type defined in the C++ standard library may be move-assigned (11.4.6 [[class.copy.assign](#)]) to itself. Such an assignment places the object in a valid but unspecified state unless otherwise specified.

2. Add a note at the end of 16.4.5.9 [[res.on.arguments](#)]/1, bullet 3, as indicated:

-1- Each of the following applies to all arguments to functions defined in the C++ standard library, unless explicitly stated otherwise.

(1.1) — [...]

(1.2) — [...]

(1.3) — If a function argument binds to an rvalue reference parameter, the implementation may assume that this parameter is a unique reference to this argument. [Note: If the parameter is a generic parameter of the form `T&&` and an lvalue of type `A` is bound, the argument binds to an lvalue reference (14.8.2.1) and thus is not covered by the previous sentence. — end note] [Note: If a program casts an lvalue to an xvalue while passing that lvalue to a library function (e.g. by calling the function with the argument `std::move(x)`), the program is effectively asking that function to treat that lvalue as a temporary. The implementation is free to optimize away aliasing checks which might be needed if the argument was an lvalue. — end note] [Note: This does not apply to the argument passed to a move assignment operator (16.4.6.16 [[lib.types.movedfrom](#)]). — end note]

3. Edit Table 83 "Container requirements" in 22.2.1 [\[container.requirements.general\]](#) as indicated:

Table 83 — Container requirements				
Expression	Return type	Operational semantics	Assertion/note pre-/post-condition	Complexity
...				
<code>a = rv</code>	<code>T&</code>	All existing elements of <code>a</code> are either move assigned to or destroyed	post: If <code>a</code> and <code>rv</code> do not refer to the same object, <code>a</code> shall be equal to the value that <code>rv</code> had before this assignment	linear
...				

4. Edit Table 86 "Allocator-aware container requirements" in 22.2.1 [\[container.requirements.general\]](#) as indicated:

Table 86 — Allocator-aware container requirements			
Expression	Return type	Assertion/note pre-/post-condition	Complexity
...			
<code>a = rv</code>	<code>T&</code>	<i>Requires:</i> If <code>allocator_traits<allocator_type>::propagate_on_container_move_assignment::value</code> is false, <code>T</code> is <code>MoveInsertable</code> into <code>x</code> and <code>MoveAssignable</code> . All existing elements of <code>a</code> are either move assigned to or destroyed. post: If <code>a</code> and <code>rv</code> do not refer to the same object, <code>a</code> shall be equal to the value that <code>rv</code> had before this assignment	linear
...			

[2018-08-16, Howard comments and provides updated wording]

I agreed to provide proposed wording for LWG 2839 that was reworded to use `MoveAssignable`. The advantage of this is that `MoveAssignable` specifies the self-assignment case, thus we do not need to repeat ourselves.

[2018-08-23 Batavia Issues processing]

Howard and Tim to discuss a revised P/R.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4762](#).

1. Add a new subsection to 16.4.6 [\[conforming\]](#) after 16.4.6.5 [\[member.functions\]](#):

[Special members \[conforming.special\]](#)

Class types defined by the C++ standard library and specified to be default constructible, move constructible, copy constructible, move assignable, copy assignable, or destructible, shall meet the associated requirements `Cpp17DefaultConstructible`, `Cpp17MoveConstructible`, `Cpp17CopyConstructible`, `Cpp17MoveAssignable`, `Cpp17CopyAssignable`, and `Cpp17Destructible`, respectively (16.4.4.2 [\[utility.arg.requirements\]](#)).

[2020-06-06 Tim restores and updates P/R following 2020-05-29 telecon discussion]

The standard doesn't define phrases like "default constructible" used in the previous P/R. Moreover, the library provides a variety of wrapper types, and whether these types meet the semantic requirements of `Cpp17Moveable` (and maybe even syntactic, depending on how "copy constructible" is interpreted) depends on the property of their underlying wrapped types, which might not even be an object type (e.g., tuple or pair of references). This is a large can of worms (see LWG [2146](#)) that we don't want to get into.

There is a suggestion in the telecon to blanket-exempt move-assignment operators from the 16.4.5.9 [\[res.on.arguments\]](#) 1.3 requirement. The revised wording below does not do so, as that would carve out not just self-move-assignment but also other aliasing scenarios in which the target object owns the source object. Whether such scenarios should be permitted is outside the scope of this issue, though notably `assignable_from` (18.4.8 [\[concept.assignable\]](#)) contains a note alluding to these cases and suggesting that they should be considered to be outside the domain of `=` entirely.

[2020-07-17; issue processing telecon]

LWG reviewed the latest proposed resolution. Unanimous consent to move to Ready.

Proposed resolution:

This wording is relative to [N4861](#).

1. Add a new paragraph at the end of 16.4.6.16 [\[lib.types.movedfrom\]](#):

-1- Objects of types defined in the C++ standard library may be moved from (`[class.copy.ctor]`). Move operations may be explicitly specified or implicitly generated. Unless otherwise specified, such moved-from objects shall be placed in a valid but unspecified state.

-?- An object of a type defined in the C++ standard library may be move-assigned (11.4.6 [class.copy.assign]) to itself. Unless otherwise specified, such an assignment places the object in a valid but unspecified state.

2. Edit 16.4.5.9 [res.on.arguments]/1, bullet 3, as indicated:

-1- Each of the following applies to all arguments to functions defined in the C++ standard library, unless explicitly stated otherwise.

(1.1) — [...]

(1.2) — [...]

(1.3) — If a function argument binds to an rvalue reference parameter, the implementation may assume that this parameter is a unique reference to this argument, except that the argument passed to a move-assignment operator may be a reference to **this* (16.4.6.16 [lib.types.movedfrom]). [Note: If the type of *a* parameter is a generic parameter of the form *T&&* and an lvalue of type *A* is bound, the argument binds to an lvalue reference (13.10.3.2 [temp.deduct.call]) and thus is not covered by the previous sentence; forwarding reference (13.10.3.2 [temp.deduct.call]) that is deduced to an lvalue reference type, then the argument is not bound to an rvalue reference. — end note] [Note: If a program casts an lvalue to an xvalue while passing that lvalue to a library function (e.g. by calling the function with the argument `std::move(x)`), the program is effectively asking that function to treat that lvalue as a temporary. The implementation is free to optimize away aliasing checks which might be needed if the argument was an lvalue. — end note]

3. Edit Table 73 "Container requirements" in 22.2.1 [container.requirements.general] as indicated:

Table 73 — Container requirements

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition	Complexity
...				
<code>a = rv</code>	<code>T&</code>	All existing elements of <code>a</code> are either move assigned to or destroyed	Postconditions: <u>If <i>a</i> and <i>rv</i> do not refer to the same object</u> , <code>a</code> is equal to the value that <code>rv</code> had before this assignment.	linear
...				

4. Edit Table 76 "Allocator-aware container requirements" in 22.2.1 [container.requirements.general] as indicated:

Table 86 — Allocator-aware container requirements

Expression	Return type	Assertion/note pre-/post-condition	Complexity
...			
<code>a = rv</code>	<code>T&</code>	Preconditions: If <code>allocator_traits<allocator_type>::propagate_on_container_move_assignment::value</code> is false, <code>T</code> is <code>Cpp17MoveInsertable</code> into <code>x</code> and <code>Cpp17MoveAssignable</code> . Effects: All existing elements of <code>a</code> are either move assigned to or destroyed. Postconditions: <u>If <i>a</i> and <i>rv</i> do not refer to the same object</u> , <code>a</code> is equal to the value that <code>rv</code> had before this assignment.	linear
...			

3117. Missing packaged_task deduction guides

Section: 32.9.10 [futures.task] Status: Ready Submitter: Marc Mutz Opened: 2018-06-08 Last modified: 2020-09-06

Priority: 3

View all other issues in [futures.task].

Discussion:

`std::function` has deduction guides, but `std::packaged_task`, which is otherwise very similar, does not. This is surprising to users and I can think of no reason for the former to be treated differently from the latter. I therefore propose to add deduction guides for packaged task with the same semantics as the existing ones for `function`.

[2018-06-23 after reflector discussion]

Priority set to 3

Previous resolution [SUPERSEDED]:

This wording is relative to [N4750](#).

1. Modify 32.9.10 [futures.task], class template `packaged_task` synopsis, as indicated:

```
namespace std {
    [...]
    template<class R, class... ArgTypes>
    class packaged_task<R(ArgTypes...)> {
        [...]
    };

    template<class R, class... ArgTypes>
    packaged_task(R (*)(ArgTypes...)) -> packaged_task<R(ArgTypes...)>;
}
```

```

template<class F> packaged_task(F) -> packaged_task<see below>;

template<class R, class... ArgTypes>
void swap(packaged_task<R(ArgTypes...)>& x, packaged_task<R(ArgTypes...)>& y) noexcept;
}

```

2. Modify 32.9.10.2 [futures.task.members] as indicated:

```

template<class F>
packaged_task(F&& f);

[...]

template<class F> packaged_task(F) -> packaged_task<see below>;

-?- Remarks: This deduction guide participates in overload resolution only if &F::operator(.) is well-formed
when treated as an unevaluated operand. In that case, if decltype(&F::operator(.)) is of the form R(G::*)(
(A...)) cv &opt_noexcept_opt for a class type G, then the deduced type is packaged_task<R(A...)>.

[...]

packaged_task(packaged_task&& rhs) noexcept;

```

[2020-02-13; Prague]

LWG improves wording matching Marshall's Mandating paper.

[2020-02-14; Prague]

Do we want a feature test macro for this new feature?

```

F N A
1 7 6

```

[Status to Ready on Friday in Prague.]

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 32.9.10 [futures.task], class template packaged_task synopsis, as indicated:

```

namespace std {
[...]
template<class R, class... ArgTypes>
class packaged_task<R(ArgTypes...)> {
[...]
};

template<class R, class... ArgTypes>
packaged_task(R (*)(ArgTypes...)) -> packaged_task<R(ArgTypes...)>;

template<class F> packaged_task(F) -> packaged_task<see below>;

template<class R, class... ArgTypes>
void swap(packaged_task<R(ArgTypes...)>& x, packaged_task<R(ArgTypes...)>& y) noexcept;
}

```

2. Modify 32.9.10.2 [futures.task.members] as indicated:

```

template<class F>
packaged_task(F&& f);

[...]

template<class F> packaged_task(F) -> packaged_task<see below>;

-?- Constraints: &F::operator(.) is well-formed when treated as an unevaluated operand and decltype(&F::operator(.)) is of
the form R(G::*)(A...)) cv &opt_noexcept_opt for a class type G.

-?- Remarks: The deduced type is packaged_task<R(A...)>.

[...]

packaged_task(packaged_task&& rhs) noexcept;

```

3143. monotonic_buffer_resource growth policy is unclear

Section: 20.12.6 [mem.res.monotonic.buffer] Status: [Ready](#) Submitter: Jonathan Wakely Opened: 2018-07-20 Last modified: 2020-07-17

Priority: 2

Discussion:

During the discussion of LWG 3120 it was pointed out that the current wording in 20.12.6 [\[mem.res.monotonic.buffer\]](#) is contradictory. The introductory text for the class says "Each additional buffer is larger than the previous one, following a geometric progression" but the spec for `do_allocate` doesn't agree.

Firstly, it's impossible for the implementation to ensure a single geometric progression, because the size of the next buffer can be arbitrarily large. If the caller asks for an allocation that is N times bigger than the previous buffer, the next buffer will be at least N times larger than the previous one. If N is larger than the implementation-defined growth factor it's not a geometric progression.

Secondly, it's not even clear that each additional buffer will be larger than the previous one. Given a `monotonic_buffer_resource` object with little remaining space in `current_buffer`, a request to allocate $10 \times \text{next_buffer_size}$ will:

"set `current_buffer` to `upstream_src->allocate(n, m)`, where n is not less than $\max(\text{bytes}, \text{next_buffer_size})$ and m is not less than `alignment`, and increase `next_buffer_size` by an implementation-defined growth factor (which need not be integral), then allocate the return block from the newly-allocated `current_buffer`."

The effects are to allocate a new buffer of at least $\max(10 \times \text{next_buffer_size}, \text{next_buffer_size})$ bytes, and then do `next_buffer_size *= growth_factor`. If `growth_factor < 10` then the next allocated buffer might be smaller than the last one. This means that although `next_buffer_size` itself follows a geometric progression, the actual size of any single allocated buffer can be much larger than `next_buffer_size`. A graph of the allocated sizes looks like a geometric progression with spikes where an allocation size is larger than `next_buffer_size`.

If the intention is to set `next_buffer_size = max(n, next_buffer_size * growth_factor)` so that every allocation from upstream is larger than the previous one, then we need a change to the *Effects*: to actually say that. Rather than a geometric progression with anomalous spikes, this would produce a number of different geometric progressions with discontinuous jumps between them.

If the spiky interpretation is right then we need to weaken the "Each additional buffer is larger" statement. Either way, we need to add a caveat to the "following a geometric progression" text because that isn't true for the spiky interpretation or the jumpy interpretation.

Thirdly, the *Effects*: says that the size of the allocated block, n , is not less than $\max(\text{bytes}, \text{next_buffer_size})$. This seems to allow an implementation to choose to do $n = \text{ceil2}(\max(\text{bytes}, \text{next_buffer_size}))$ if it wishes (maybe because allocating sizes that are a power of 2 simplifies the `monotonic_buffer_resource` implementation, or allows reducing the bookkeeping overhead). This still results in an approximate geometric progression (under either the spiky or jumpy interpretation) but the graph has steps rather than being a smooth curve (but always above the curve). This is another way that "Each additional buffer is larger than the previous one" is not guaranteed. Even if $\max(\text{bytes}, \text{next_buffer_size})$ is greater on every call, for a growth factor between 1.0 and 2.0 the result of `ceil2` might be the same for two successive buffers. I see no reason to forbid this, but Pablo suggested it's not allowed because it doesn't result in exponential growth (which I disagree with). If this is supposed to be forbidden, the wording needs to be fixed to forbid it.

[2019-01-20 Reflector prioritization]

Set Priority to 2

[2020-02-13, Prague]

LWG looked at the issue and a suggestion was presented to eliminate most of 20.12.6 [\[mem.res.monotonic.buffer\]](#) to solve the problem the current requirements impose.

[2020-02-16; Prague]

Reviewed revised wording and moved to Ready for Varna.

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 20.12.6 [\[mem.res.monotonic.buffer\]](#), as indicated:

-1- A `monotonic_buffer_resource` is a special-purpose memory resource intended for very fast memory allocations in situations where memory is used to build up a few objects and then is released all at once when the memory resource object is destroyed. ~~It has the following qualities:~~

~~(1.1) — A call to `deallocate` has no effect, thus the amount of memory consumed increases monotonically until the resource is destroyed.~~

~~(1.2) — The program can supply an initial buffer, which the allocator uses to satisfy memory requests.~~

~~(1.3) — When the initial buffer (if any) is exhausted, it obtains additional buffers from an upstream memory resource supplied at construction. Each additional buffer is larger than the previous one, following a geometric progression.~~

~~(1.4) — It is intended for access from one thread of control at a time. Specifically, calls to `allocate` and `deallocate` do not synchronize with one another.~~

~~(1.5) — It frees the allocated memory on destruction, even if `deallocate` has not been called for some of the allocated blocks.~~

3195. What is the stored pointer value of an empty `weak_ptr`?

Section: 20.11.4.2 [\[util.smartptr.weak.const\]](#) Status: [Ready](#) Submitter: Casey Carter Opened: 2019-03-15 Last modified: 2020-09-06

Priority: 2

Discussion:

20.11.4.2 [\[util.smartptr.weak.const\]](#) specifies `weak_ptr`'s default constructor:

```
constexpr weak_ptr() noexcept;
```

1 *Effects*: Constructs an empty `weak_ptr` object.

2 *Ensures*: `use_count() == 0`.

and `shared_ptr` converting constructor template:

```
weak_ptr(const weak_ptr& r) noexcept;  
template<class Y> weak_ptr(const weak_ptr<Y>& r) noexcept;  
template<class Y> weak_ptr(const shared_ptr<Y>& r) noexcept;
```

3 *Remarks*: The second and third constructors shall not participate in overload resolution unless Y^* is compatible with T^* .

4 *Effects*: If `r` is empty, constructs an empty `weak_ptr` object; otherwise, constructs a `weak_ptr` object that shares ownership with `r` and stores a copy of the pointer stored in `r`.

5 *Ensures*: `use_count() == r.use_count()`.

Note that neither specifies the value of the stored pointer when the resulting `weak_ptr` is empty. This didn't matter — the stored pointer value was unobservable for an empty `weak_ptr` — until we added `atomic<weak_ptr>`. 31.8.7.3 [util.smartptr.atomic.weak]/15 says:

Remarks: Two `weak_ptr` objects are equivalent if they store the same pointer value and either share ownership, or both are empty. The weak form may fail spuriously. See 31.8.2 [atomics.types.operations].

Two empty `weak_ptr` objects that store different pointer values are not equivalent. We *could* correct this by changing 31.8.7.3 [util.smartptr.atomic.weak]/15 to "Two `weak_ptr` objects are equivalent if they are both empty, or if they share ownership and store the same pointer value." In practice, an implementation of `atomic<weak_ptr>` will CAS on both the ownership (control block pointer) and stored pointer value, so it seems cleaner to pin down the stored pointer value of an empty `weak_ptr`.

[2019-06-09 Priority set to 2 after reflector discussion]

Previous resolution [SUPERSEDED]

This wording is relative to [N4810](#).

1. Modify 20.11.4.2 [util.smartptr.weak.const] as indicated (note the drive-by edit to cleanup the occurrences of "constructs an object of class `foo`");

```
constexpr weak_ptr() noexcept;
```

-1- *Effects*: Constructs an empty `weak_ptr` object that stores a null pointer value.

-2- *Ensures*: `use_count() == 0`.

```
weak_ptr(const weak_ptr& r) noexcept;  
template<class Y> weak_ptr(const weak_ptr<Y>& r) noexcept;  
template<class Y> weak_ptr(const shared_ptr<Y>& r) noexcept;
```

-3- *Remarks*: The second and third constructors shall not participate in overload resolution unless Y^* is compatible with T^* .

-4- *Effects*: If `r` is empty, constructs an empty `weak_ptr` object that stores a null pointer value; otherwise, constructs a `weak_ptr` object that shares ownership with `r` and stores a copy of the pointer stored in `r`.

-5- *Ensures*: `use_count() == r.use_count()`.

[2020-02-14 Casey updates P/R per LWG instruction]

While reviewing the P/R in Prague, Tim Song noticed that the stored pointer value of a moved-from `weak_ptr` must also be specified.

[2020-02-16; Prague]

Reviewed revised wording and moved to Ready for Varna.

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 20.11.4.2 [util.smartptr.weak.const] as indicated:

```
constexpr weak_ptr() noexcept;
```

-1- *Effects*: Constructs an empty `weak_ptr` object that stores a null pointer value.

-2- *Postconditions*: `use_count() == 0`.

```
weak_ptr(const weak_ptr& r) noexcept;  
template<class Y> weak_ptr(const weak_ptr<Y>& r) noexcept;  
template<class Y> weak_ptr(const shared_ptr<Y>& r) noexcept;
```

-3- *Remarks:* The second and third constructors shall not participate in overload resolution unless Υ^* is compatible with τ^* .

-4- *Effects:* If r is empty, constructs an empty `weak_ptr` object that stores a null pointer value; otherwise, constructs a `weak_ptr` object that shares ownership with r and stores a copy of the pointer stored in r .

-5- *Postconditions:* `use_count() == r.use_count()`.

```
weak_ptr(weak_ptr&& r) noexcept;  
template<class Y> weak_ptr(weak_ptr<Y>&& r) noexcept;
```

-6- *Remarks:* The second constructor shall not participate in overload resolution unless Υ^* is compatible with τ^* .

-7- *Effects:* Move constructs a `weak_ptr` instance from r .

-8- *Postconditions:* `*this` ~~shall contain~~contains the old value of r . r ~~shall be~~is empty, stores a null pointer value, and `r.use_count() == 0`.

3211. `std::tuple<>` should be trivially constructible

Section: 20.5.3.1 [[tuple.cnstr](#)] **Status:** [Ready](#) **Submitter:** Louis Dionne **Opened:** 2019-05-29 **Last modified:** 2020-09-06

Priority: 3

View other [active issues](#) in [[tuple.cnstr](#)].

View all other [issues](#) in [[tuple.cnstr](#)].

Discussion:

That requirement is really easy to enforce, and it has been requested by users (e.g. [libc++ bug 41714](#)).

Previous resolution [SUPERSEDED]:

This wording is relative to [N4810](#).

1. Modify 20.5.3.1 [[tuple.cnstr](#)] as indicated:

-4- If `is_trivially_destructible_v< $\tau_i$ >` is true for all τ_i , then the destructor of `tuple` is trivial. The default constructor of `tuple<>` is trivial.

[2020-02-13, Prague]

LWG discussion revealed that all were happy that we want this, except that the new wording should become a separate paragraph.

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 20.5.3.1 [[tuple.cnstr](#)] as indicated:

-4- If `is_trivially_destructible_v< $\tau_i$ >` is true for all τ_i , then the destructor of `tuple` is trivial.

-?- The default constructor of `tuple<>` is trivial.

3236. Random access iterator requirements lack limiting relational operators domain to comparing those from the same range

Section: 23.3.5.7 [[random.access.iterators](#)] **Status:** [Ready](#) **Submitter:** Peter Sommerlad **Opened:** 2019-07-15 **Last modified:** 2020-09-06

Priority: 3

View all other [issues](#) in [[random.access.iterators](#)].

Discussion:

For forward iterators we have very clear wording regarding the restricted domain of `operator==` in 23.3.5.5 [[forward.iterators](#)] p2:

The domain of `==` for forward iterators is that of iterators over the same underlying sequence. However, value-initialized iterators may be compared and shall compare equal to other value-initialized iterators of the same type. [Note: Value-initialized iterators behave as if they refer past the end of the same empty sequence. — *end note*]

But for the relational operators of random access iterators specified in 23.3.5.7 [[random.access.iterators](#)], Table [tab:randomaccessiterator], no such domain constraints are clearly defined, except that we can infer that they are similarly constrained as the difference of the compared iterators by means of the operational semantics of `operator<`.

[2019-07-29; Casey comments and provides wording]

Change the "Operational Semantics" column of the "a < b" row of [tab:randomaccessiterator] to "**Effects: Equivalent to: return** b - a > 0;

It then follows that a < b is required to be well-defined over the domain for which b - a is required to be well-defined, which is the set of pairs (x, y) such that there exists a value n of type difference_type such that x + n == b.

[2020-02-13, Prague]

P3, but some hesitation to make it Immediate, therefore moving to Ready.

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 23.3.5.7 [\[random.access.iterators\]](#) as indicated:

Table 87: Cpp17RandomAccessIterator requirements (in addition to Cpp17BidirectionalIterator)
[tab:randomaccessiterator]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
[...]			
a < b	contextually convertible to bool	Effects: Equivalent to: return b - a > 0;	< is a total ordering relation

3249. There are no 'pointers' in §[atomics.lockfree]

Section: 31.5 [\[atomics.lockfree\]](#) **Status:** [Ready](#) **Submitter:** Billy O'Neal III **Opened:** 2019-08-03 **Last modified:** 2020-09-06

Priority: 4

View all other [issues](#) in [atomics.lockfree].

Discussion:

According to SG1 experts, the requirement in [atomics.lockfree]/2 is intended to require that the answer for is_lock_free() be the same for a given T for a given run of the program. The wording does not achieve that because it's described in terms of 'pointers', but there are no pointers in an atomic<int>.

[2020-02 Status to Ready on Thursday morning in Prague.]

Proposed resolution:

This wording is relative to [N4830](#).

1. Modify 31.5 [\[atomics.lockfree\]](#) as indicated:

-2- The functions **s_atomic<T>::is lock free, and** atomic_is_lock_free (31.8.2 [\[atomics.types.operations\]](#)) indicates whether the object is lock-free. In any given program execution, the result of the lock-free query **is the same for all atomic objects** ~~shall be consistent for all pointers~~ of the same type.

3265. move_iterator's conversions are more broken after P1207

Section: 23.5.3.4 [\[move.iter.cons\]](#) **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2019-08-23 **Last modified:** 2020-09-06

Priority: 2

View other [active issues](#) in [move.iter.cons].

View all other [issues](#) in [move.iter.cons].

Discussion:

The converting constructor and assignment operator specified in 23.5.3.4 [\[move.iter.cons\]](#) were technically broken before P1207:

- 23.5.3.4 [\[move.iter.cons\]](#) para 3 (and 5 for that matter) is an instance of LWG [3105](#); it should instead mandate that u.base() is convertible to Iterator.
- 23.5.3.4 [\[move.iter.cons\]](#) para 5 uses "is convertible" to guard an assignment operation instead of a conversion; it should instead mandate.

After applying [P1207R4](#) "Movability of Single-pass Iterators", u.base() is not always well-formed, exacerbating the problem. These operations must ensure that u.base() is well-formed.

Drive-by:

- Let's burninate "Constructs a move_iterator" while we're touching this subclause.
- We'll also burninate "Iterator operations applied..." since the requirement it wants to impose is covered (and indeed must be covered) by the specifications of those other operations.

[2019-09-14 Priority set to 2 based on reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4830](#).

1. Modify 23.5.3.4 [\[move.iter.cons\]](#) as indicated:

```
constexpr move_iterator();
```

-1- Effects: ~~Constructs a move_iterator, v~~Value-initializes current. ~~Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type Iterator.~~

```
constexpr explicit move_iterator(Iterator i);
```

-2- Effects: ~~Constructs a move_iterator, i~~Initializes current with `std::move(i)`.

```
template<class U> constexpr move_iterator(const move_iterator<U>& u);
```

-3- Mandates: ~~U~~`u.base()` is well-formed and convertible to `Iterator`.

-4- Effects: ~~Constructs a move_iterator, i~~Initializes current with `u.base()`.

```
template<class U> constexpr move_iterator& operator=(const move_iterator<U>& u);
```

-5- Mandates: ~~U is convertible to Iterator~~`u.base()` is well-formed and is assignable `v<Iterator&, const U&>` is true.

-6- Effects: Assigns `u.base()` to current.

[2020-02-14; Prague]

LWG Review. Some wording improvements have been made and lead to revised wording.

[2020-02-16; Prague]

Reviewed revised wording and moved to Ready for Varna.

[2020-07-17; superseded by [3435](#)]

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 23.5.3.4 [\[move.iter.cons\]](#) as indicated:

```
constexpr move_iterator();
```

-1- Effects: ~~Constructs a move_iterator, v~~Value-initializes current. ~~Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type Iterator.~~

```
constexpr explicit move_iterator(Iterator i);
```

-2- Effects: ~~Constructs a move_iterator, i~~Initializes current with `std::move(i)`.

```
template<class U> constexpr move_iterator(const move_iterator<U>& u);
```

-3- Mandates: ~~U~~`u.base()` is well-formed and convertible to `Iterator`.

-4- Effects: ~~Constructs a move_iterator, i~~Initializes current with `u.base()`.

```
template<class U> constexpr move_iterator& operator=(const move_iterator<U>& u);
```

-5- Mandates: ~~U is convertible to Iterator~~`u.base()` is well-formed and is assignable `v<Iterator&, U>` is true.

-6- Effects: Assigns `u.base()` to current.

[3432](#). Missing requirement for `comparison_category`

Section: 21.4.5 [\[string.view.comparison\]](#) Status: [Ready](#) Submitter: Jonathan Wakely Opened: 2020-04-19 Last modified: 2020-09-06

Priority: 0

Discussion:

It's not clear what happens if a program-defined character traits type defines `comparison_category` as a synonym for `void`, or some other bogus type.

Discussion on the LWG reflector settled on making it ill-formed at the point of use.

[2020-07-17; Moved to Ready in telecon]

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 21.4.5 [\[string.view.comparison\]](#) by adding a new paragraph after p3:

As result of reflector discussion we decided to make a drive-by fix in p3 below.

```
template<class charT, class traits>
constexpr see below operator<=>(basic_string_view<charT, traits> lhs,
                                basic_string_view<charT, traits> rhs) noexcept;
```

-3- Let R denote the type `traits::comparison_category` if [that qualified-id is valid and denotes a type \(13.10.3 \[temp.deduct\]\)](#) ~~exists~~, otherwise R is `weak_ordering`.

~~?- Mandates: R denotes a comparison category type (17.11.2 [cmp.categories]).~~

-4- Returns: `static_cast<R>(lhs.compare(rhs) <=> 0)`.

[3443](#). [\[networking.ts\]](#) `net::basic_socket_iostream` should use `addressof`

Section: 19.2.1 [\[networking.ts::socket.iostream.cons\]](#) Status: [Ready](#) Submitter: Jonathan Wakely Opened: 2020-05-14 Last modified: 2020-07-17

Priority: 0

View other [active issues](#) in [\[networking.ts::socket.iostream.cons\]](#).

View all other [issues](#) in [\[networking.ts::socket.iostream.cons\]](#).

Discussion:

Addresses: [networking.ts](#)

19.2.1 [\[networking.ts::socket.iostream.cons\]](#) uses `&sb_` which could find something bad via ADL.

[2020-07-17; Moved to Ready in telecon]

Jens suggested we should have blanket wording saying that when the library uses the `&` operator, it means `std::addressof`.

Proposed resolution:

This wording is relative to [N4771](#).

1. Modify 19.2.1 [\[networking.ts::socket.iostream.cons\]](#) as indicated:

```
basic_socket_iostream();
```

-1- Effects: Initializes the base class as `basic_iostream<char>(&addressof(sb_))`, value-initializes `sb_`, and performs `setf(std::ios_base::unitbuf)`.

```
explicit basic_socket_iostream(basic_stream_socket<protocol_type> s);
```

-2- Effects: Initializes the base class as `basic_iostream<char>(&addressof(sb_))`, initializes `sb_` with `std::move(s)`, and performs `setf(std::ios_base::unitbuf)`.

```
basic_socket_iostream(basic_socket_iostream&& rhs);
```

-3- Effects: Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_socket_streambuf`. Next `basic_iostream<char>::set_rdbuf(&addressof(sb_))` is called to install the contained `basic_socket_streambuf`.

```
template<class... Args>
explicit basic_socket_iostream(Args&&... args);
```

-4- Effects: Initializes the base class as `basic_iostream<char>(&addressof(sb_))`, value-initializes `sb_`, and performs `setf(std::ios_base::unitbuf)`. Then calls `rdbuf()->connect(forward<Args>(args)...)`. If that function returns a null pointer, calls `setstate(failbit)`.

[3447](#). Deduction guides for `take_view` and `drop_view` have different constraints

Section: 24.7.7.2 [\[range.take.view\]](#) Status: [Ready](#) Submitter: Jens Maurer Opened: 2020-05-15 Last modified: 2020-07-17

Priority: 0

View all other [issues](#) in [\[range.take.view\]](#).

Discussion:

From this [editorial issue request](#):

(Note "range R" vs "class R".)

In 24.7.7.2 [\[range.take.view\]](#), the deduction guide for take_view is declared as:

```
template<range R>
take_view(R&&, range_difference_t<R>)
-> take_view<views::all_t<R>>;
```

In 24.7.9.2 [\[range.drop.view\]](#), the deduction guide for drop_view is declared as:

```
template<class R>
drop_view(R&&, range_difference_t<R>) -> drop_view<views::all_t<R>>;
```

Note the difference between their template parameter lists.

Suggested resolution:

Change the deduction guide of take_view from

```
template<range R>
```

to

```
template<class R>
```

[2020-07-17; Moved to Ready in telecon]

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 24.7.7.2 [\[range.take.view\]](#), class template take_view synopsis, as indicated:

```
[...]
template<rangeclass R>
take_view(R&&, range_difference_t<R>)
-> take_view<views::all_t<R>>;
[...]
```

[3450](#). The const overloads of take_while_view::begin/end are underconstrained

Section: 24.7.8.2 [\[range.take.while.view\]](#) Status: [Ready](#) Submitter: Tim Song Opened: 2020-06-06 Last modified: 2020-07-17

Priority: 0

View all other [issues](#) in [\[range.take.while.view\]](#).

Discussion:

The const overloads of take_while_view::begin and take_while_view::end are underconstrained: they should require the predicate to model indirect_unary_predicate<iterator_t<const V>>. A simple example is

```
#include <ranges>

auto v = std::views::single(1) | std::views::take_while([](int& x) { return true;});
static_assert(std::ranges::range<decltype(v)>);
static_assert(std::ranges::range<decltype(v) const>);
bool b = std::ranges::cbegin(v) == std::ranges::cend(v);
```

Here, the static_asserts pass but the comparison fails to compile. The other views using indirect_unary_predicate (filter_view and drop_while_view) do not have this problem because they do not support const-iteration.

[2020-07-17; Moved to Ready in telecon]

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 24.7.8.2 [\[range.take.while.view\]](#), class template take_while_view synopsis, as indicated:

```
namespace std::ranges {
    template<view V, class Pred>
        requires input_range<V> && is_object_v<Pred> &&
            indirect_unary_predicate<const Pred, iterator_t<V>>
    class take_while_view : public view_interface<take_while_view<V, Pred>> {
    [...]

    constexpr auto begin() requires (!simple-view<V>)
    { return ranges::begin(base_); }

    constexpr auto begin() const requires range<const V> && indirect_unary_predicate<const Pred, iterator_t<const V>>
    { return ranges::begin(base_); }
```

```
constexpr auto end() requires (!simple-view<V>)
{ return sentinel<false>(ranges::end(base_), addressof(*pred_)); }

constexpr auto end() const requires range<const V> && indirect_unary_predicate<const Pred, iterator_t<const V>>
{ return sentinel<true>(ranges::end(base_), addressof(*pred_)); }
};
}
```

3464. istream::gcount() can overflow

Section: 29.7.4.4 [\[istream.unformatted\]](#) **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2020-07-10 **Last modified:** 2020-09-06

Priority: 0

View all other [issues](#) in [\[istream.unformatted\]](#).

Discussion:

The standard doesn't say what `gcount()` should return if the last unformatted input operation extracted more than `numeric_limits<streamsize>::max()` characters. This is possible when using `istream::ignore(numeric_limits<streamsize>::max(), delim)`, which will keep extracting characters until the delimiter is found. On a 32-bit platform files larger than 2GB can overflow the counter, so can a streambuf reading from a network socket, or producing random characters.

Libstdc++ saturates the counter in `istream::ignore`, so that `gcount()` returns `numeric_limits<streamsize>::max()`. Libc++ results in an integer overflow.

As far as I'm aware, only `istream::ignore` can extract more than `numeric_limits<streamsize>::max()` characters at once. We could either fix it in the specification of `ignore`, or in `gcount`.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

Option A:

1. Modify 29.7.4.4 [\[istream.unformatted\]](#) as indicated:

```
streamsize gcount() const;
```

-2- *Effects*: None. This member function does not behave as an unformatted input function (as described above).

-3- *Returns*: The number of characters extracted by the last unformatted input member function called for the object. If the number cannot be represented, returns `numeric_limits<streamsize>::max()`.

Option B:

1. Modify 29.7.4.4 [\[istream.unformatted\]](#) as indicated:

```
basic_istream<charT, traits>& ignore(streamsize n = 1, int_type delim = traits::eof());
```

-25- *Effects*: Behaves as an unformatted input function (as described above). After constructing a sentry object, extracts characters and discards them. Characters are extracted until any of the following occurs:

(25.1) — `n != numeric_limits<streamsize>::max()` (17.3.5 [\[numeric.limits\]](#)) and `n` characters have been extracted so far

(25.2) — end-of-file occurs on the input sequence (in which case the function calls `setstate(eofbit)`, which may throw `ios_base::failure` (29.5.5.4 [\[iostate.flags\]](#)));

(25.3) — `traits::eq_int_type(traits::to_int_type(c), delim)` for the next available input character `c` (in which case `c` is extracted).

-?- If the number of characters extracted is greater than `numeric_limits<streamsize>::max()` then for the purposes of `gcount()` the number is treated as `numeric_limits<streamsize>::max()`.

-26- *Remarks*: The last condition will never occur if `traits::eq_int_type(delim, traits::eof())`.

-27- *Returns*: `*this`.

[2020-07-17; Moved to Ready in telecon]

On the reflector Davis pointed out that there are other members which can cause `gcount()` to overflow. There was unanimous agreement on the reflector and the telecon that Option A is better.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 29.7.4.4 [\[istream.unformatted\]](#) as indicated:

```
streamsize gcount() const;
```

-2- *Effects*: None. This member function does not behave as an unformatted input function (as described above).

-3- *Returns*: The number of characters extracted by the last unformatted input member function called for the object. **If the number cannot be represented, returns numeric_limits<streamsize>::max().**

Tentatively Ready Issues

These issues were moved to **Tentatively Ready** since the Prague meeting, either during LWG teleconferences or following reflector polls.

2731. Existence of `lock_guard<MutexTypes...>::mutex_type` typedef unclear

Section: 32.5.5.2 [[thread.lock.guard](#)] **Status:** [Tentatively Ready](#) **Submitter:** Eric Fiselier **Opened:** 2016-06-13 **Last modified:** 2020-05-16

Priority: 3

View all other [issues](#) in [[thread.lock.guard](#)].

Discussion:

In the synopsis of 32.5.5.3 [[thread.lock.scoped](#)] the `mutex_type` typedef is specified as follows:

```
template <class... MutexTypes>
class scoped_lock {
public:
    typedef Mutex mutex_type; // If MutexTypes... consists of the single type Mutex
    [...]
};
```

The comment seems ambiguous as it could mean either:

1. `sizeof...(MutexTypes) == 1`.
2. `sizeof...(MutexTypes) >= 1` and every type in `MutexTypes...` is the same type.

I originally took the language to mean (2), but upon further review it seems that (1) is the intended interpretation, as suggested in the LEWG discussion in [Lenexa](#).

I think the language should be clarified to prevent implementation divergence.

[2016-07, Toronto Saturday afternoon issues processing]

General feeling that `sizeof...(MutexTypes) == 1` is a better way to state the requirement.

Reworked the text to refer to `scoped_lock` instead of `lock_guard`

Marshall and Eric to reword and discuss on reflector. Status to Open

[2018-3-14 Wednesday evening issues processing; general agreement to adopt once the wording is updated.]

2018-03-18 Marshall provides updated wording.

Previous resolution: [SUPERSEDED]

This wording is relative to N4594.

1. Edit 32.5.5.2 [[thread.lock.guard](#)]/1, class template `lock_guard` synopsis, as indicated:

```
template <class... MutexTypes>
class lock_guard {
public:
    typedef Mutex mutex_type; // Only if MutexTypes... consists of the expands to a single type Mutex
    [...]
};
```

Previous resolution: [SUPERSEDED]

This wording is relative to N4727.

1. Edit 32.5.5.2 [[thread.lock.guard](#)]/1, class template `lock_guard` synopsis, as indicated:

```
template <class... MutexTypes>
class scoped_lock {
public:
    using mutex_type = Mutex; // Only if sizeof...(MutexTypes) == 1 MutexTypes... consists of the single type Mutex
    [...]
};
```

[2020-05-11; Daniel provides improved wording]

[2020-05-16 Reflector discussions]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4861](#).

1. Edit 32.5.5.3 [\[thread.lock.scoped\]](#), class template `scoped_lock` synopsis, as indicated:

```
template <class... MutexTypes>
class scoped_lock {
public:
    using mutex_type = Mutex; // If MutexTypes... consists of the single type Mutex
    using mutex_type = see below; // Only if sizeof...(MutexTypes) == 1
    [...]
};
```

-1- An object of type `scoped_lock` controls the ownership of lockable objects within a scope. A `scoped_lock` object maintains ownership of lockable objects throughout the `scoped_lock` object's lifetime (6.7.3 [\[basic.life\]](#)). The behavior of a program is undefined if the lockable objects referenced by `pm` do not exist for the entire lifetime of the `scoped_lock` object.

- When `sizeof...(MutexTypes)` is `1`, let `Mutex` denote the sole type constituting the pack `MutexTypes`; the supplied `Mutex` type shall meet the `Cpp17BasicLockable` requirements (32.2.5.2 [\[thread.req.lockable.basic\]](#)). The member typedef-name `mutex_type` denotes the same type as `Mutex`.
- Otherwise, each of the mutex types all types in the template parameter pack `MutexTypes` shall meet the `Cpp17Lockable` requirements (32.2.5.3 [\[thread.req.lockable.req\]](#)) and there is no member `mutex_type`.

2743. p0083r3 `node_handle` private members missing "exposition only" comment

Section: 22.2.4.1 [\[container.node.overview\]](#) Status: [Tentatively Ready](#) Submitter: Richard Smith Opened: 2016-07-08 Last modified: 2020-05-16

Priority: 3

View other [active issues](#) in [\[container.node.overview\]](#).

View all other [issues](#) in [\[container.node.overview\]](#).

Discussion:

The private members of `node_handle` are missing the usual "exposition only" comment. As a consequence, `ptr_` and `alloc_` now appear to be names defined by the library (so programs defining these names as macros before including a library header have undefined behavior).

Presumably this is unintentional and these members should be considered to be for exposition only.

It's also not clear whether the name `node_handle` is reserved for library usage or not; 22.2.4.1 [\[container.node.overview\]](#)/3 says the implementation need not provide a type with this name, but doesn't seem to rule out the possibility that an implementation will choose to do so regardless.

Daniel:

A similar problem seems to exist for the exposition-only type `call_wrapper` from [p0358r1](#), which exposes a private data member named `fd` and a typedef `FD`.

[2016-07 Chicago]

Jonathan says that we need to make clear that the name `node_handle` is not reserved

[2019-03-17; Daniel comments and provides wording]

Due to an editorial step, the previous name `node_handle/node_handle` has been replaced by the artificial `node-handle` name, so I see no longer any reason to talk about a name `node_handle` reservation. The provided wording therefore only takes care of the private members.

[2020-05-16 Reflector discussions]

Status to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4810](#).

1. Change 22.2.4.1 [\[container.node.overview\]](#), exposition-only class template `node-handle` synopsis, as indicated:

```
template<unspecified>
class node-handle {
public:
    [...]
private:
    using container_node_type = unspecified; // exposition only
    using ator_traits = allocator_traits<allocator_type>; // exposition only
```

```

    typename ator_traits::template rebound_traits<container_node_type>::pointer ptr_; // exposition only
    optional<allocator_type> alloc_; // exposition only

public:
    [...]
};

```

2820. Clarify <cstdint> macros

Section: 17.4 [\[cstdint\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Thomas Köppe **Opened:** 2016-11-12 **Last modified:** 2020-09-06

Priority: 3

View all other [issues](#) in [cstdint].

Discussion:

I would like clarification from LWG regarding the various limit macros like INT_8_MIN in <cstdint>, in pursuit of editorial cleanup of this header's synopsis. I have two questions:

1. At present, macros like INT_8_MIN that correspond to the optional type int8_t are required (unconditionally), whereas the underlying type to which they appertain is only optional. Is this deliberate? Should the macros also be optional?
2. Is it deliberate that C++ only specifies sized aliases for the sizes 8, 16, 32 and 64, whereas the corresponding C header allows type aliases and macros for arbitrary sizes for implementations that choose to provide extended integer types? Is the C++ wording more restrictive by accident?

[2017-01-27 Telecon]

Priority 3

[2017-03-04, Kona]

C11 ties the macro names to the existence of the types. Marshall to research the second question.

Close [2764](#) as a duplicate of this issue.

[2017-03-18, Thomas comments and provides wording]

This is as close as I can get to the C wording without resolving part (a) of the issue (whether we deliberately don't allow sized type aliases for sizes other than 8, 16, 32, 64, a departure from C). Once we resolve part (a), we need to revisit <stdint.h> and fix up the synopsis (perhaps to get rid of N) and add similar wording as the one below to make the formatting macros for the fixed-width types optional. For historical interest, this issue is related to LWG [553](#) and LWG [841](#).

[2016-07, Toronto Saturday afternoon issues processing]

Status to Open

Previous resolution: [SUPERSEDED]

This wording is relative to [N4640](#).

1. Append the following content to 17.4.2 [\[cstdint.syn\]](#) p2:

-2- The header defines all types and macros the same as the C standard library header <stdint.h>. *In particular, for each of the fixed-width types (int8_t, int16_t, int32_t, int64_t, uint8_t, uint16_t, uint32_t, uint64_t) the type alias and the corresponding limit macros are defined if and only if the implementation provides the corresponding type.*

[2017-10-21, Thomas Köppe provides improved wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4687](#).

1. Change 17.4.2 [\[cstdint.syn\]](#), header <cstdint> synopsis, as indicated:

```

[...
using int64_t = signed integer type; // optional
using intN_t = see below; // optional, see below
[...
using int_fast64_t = signed integer type;
using int_fastN_t = see below; // optional, see below
[...
using int_least64_t = signed integer type;
using int_leastN_t = see below; // optional, see below
[...
using uint64_t = unsigned integer type; // optional
using uintN_t = see below; // optional, see below
[...
using uint_fast64_t = unsigned integer type;

```

```

using uint_fastN_t = see below; // optional, see below
[...]
using uint_least64_t = unsigned integer type;
using uint_leastN_t = see below; // optional, see below

using uintmax_t = unsigned integer type;
using uintptr_t = unsigned integer type; // optional

#define INT_N_MIN see below;
#define INT_N_MAX see below;
#define UINT_N_MAX see below;

#define INT_FASTN_MIN see below;
#define INT_FASTN_MAX see below;
#define UINT_FASTN_MAX see below;

#define INT_LEASTN_MIN see below;
#define INT_LEASTN_MAX see below;
#define UINT_LEASTN_MAX see below;

#define INTMAX_MIN see below;
#define INTMAX_MAX see below;
#define UINTMAX_MAX see below;

#define INTPTR_MIN see below;
#define INTPTR_MAX see below;
#define UINTPTR_MAX see below;

#define PTRDIFF_MIN see below;
#define PTRDIFF_MAX see below;
#define SIZE_MAX see below;

#define SIGATOMIC_MIN see below;
#define SIGATOMIC_MAX see below;

#define WCHAR_MIN see below;
#define WCHAR_MAX see below;

#define WINT_MIN see below;
#define WINT_MAX see below;

#define INTN_C(value) see below;
#define UINTN_C(value) see below;
#define INTMAX_C(value) see below;
#define UINTMAX_C(value) see below;

```

-1- The header also defines numerous macros of the form:

```

INT_{FAST LEAST}{8 16 32 64}_MIN
[UINT_{FAST LEAST}{8 16 32 64}_MAX
INT_{MAX_PTR}_MIN
[UINT_{MAX_PTR}_MAX
{PTRDIFF SIG_ATOMIC_WCHAR_WINT}_{_MAX _MIN}
SIZE_MAX

```

plus function macros of the form:

```

[U]INT{8 16 32 64 MAX}_C

```

-2- The header defines all types and macros the same as the C standard library header `<stdint.h>`. SEE ALSO: ISO C 7.20

-?- In particular, all types that use the placeholder *N* are optional when *N* is not 8, 16, 32 or 64. The exact-width types `intN_t` and `uintN_t` for *N* = 8, 16, 32, 64 are also optional; however, if an implementation provides integer types with the corresponding width, no padding bits, and (for the signed types) that have a two's complement representation, it defines the corresponding typedef names. Only those macros are defined that correspond to typedef names that the implementation actually provides. [Note: The macros `INTN_C` and `UINTN_C` correspond to the typedef names `int_leastN_t` and `uint_leastN_t`, respectively. — end note]

2. Change 29.12.2 [[cinttypes.syn](#)] as indicated:

```

#define PRIdNN see below
#define PRIiNN see below
#define PRIOiNN see below
#define PRIuNN see below
#define PRIdNN see below
#define PRIxNN see below
#define SCNdNN see below
#define SCNiNN see below
#define SCNoNN see below
#define SCNuNN see below
#define SCNxNN see below
#define PRIdLEASTNN see below
#define PRIiLEASTNN see below
#define PRIoLEASTNN see below
#define PRIuLEASTNN see below
#define PRIdLEASTNN see below
#define PRIxLEASTNN see below
#define SCNdLEASTNN see below
#define SCNiLEASTNN see below
#define SCNoLEASTNN see below

```



```

#define SCNuLEASTN see below
#define SCNxLEASTN see below
#define PRIdFASTN see below
#define PRIiFASTN see below
#define PRIoFASTN see below
#define PRIuFASTN see below
#define PRIxFASTN see below
#define PRIfFASTN see below
#define SCNdFASTN see below
#define SCNiFASTN see below
#define SCNoFASTN see below
#define SCNuFASTN see below
#define SCNxFASTN see below
[...]
```

-1- The contents and meaning of the header <stdintypes> [...]

-2- In particular, macros that use the placeholder *N* are defined if and only if the implementation actually provides the corresponding typedef name in 17.4.2 [cstdint.syn], and moreover, the fscanf macros are provided unless the implementation does not have a suitable fscanf length modifier for the type.

[2018-04-03; Geoffrey Romer suggests improved wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4727](#).

1. Change 17.4.2 [cstdint.syn], header <stdint> synopsis, as indicated:

```

[...]
```

```

using int64_t = signed integer type; // optional
using intN_t = see below; // optional, see below
[...]
```

```

using int_fast64_t = signed integer type;
using int_fastN_t = see below; // optional, see below
[...]
```

```

using int_least64_t = signed integer type;
using int_leastN_t = see below; // optional, see below
[...]
```

```

using uint64_t = unsigned integer type; // optional
using uintN_t = see below; // optional, see below
[...]
```

```

using uint_fast64_t = unsigned integer type;
using uint_fastN_t = see below; // optional, see below
[...]
```

```

using uint_least64_t = unsigned integer type;
using uint_leastN_t = see below; // optional, see below

using uintmax_t = unsigned integer type;
using uintptr_t = unsigned integer type; // optional

#define INT_N_MIN see below;
#define INT_N_MAX see below;
#define UINT_N_MAX see below;

#define INT_FASTN_MIN see below;
#define INT_FASTN_MAX see below;
#define UINT_FASTN_MAX see below;

#define INT_LEASTN_MIN see below;
#define INT_LEASTN_MAX see below;
#define UINT_LEASTN_MAX see below;

#define INTMAX_MIN see below;
#define INTMAX_MAX see below;
#define UINTMAX_MAX see below;

#define INTPTR_MIN see below;
#define INTPTR_MAX see below;
#define UINTPTR_MAX see below;

#define PTRDIFF_MIN see below;
#define PTRDIFF_MAX see below;
#define SIZE_MAX see below;

#define SIGATOMIC_MIN see below;
#define SIGATOMIC_MAX see below;

#define WCHAR_MIN see below;
#define WCHAR_MAX see below;

#define WINT_MIN see below;
#define WINT_MAX see below;

#define INTN_C(value) see below;
#define UINTN_C(value) see below;
#define INTMAX_C(value) see below;
#define UINTMAX_C(value) see below;
```

-1- The header also defines numerous macros of the form:

```
INT_[FAST LEAST]{8 16 32 64}_MIN  
[U]INT_[FAST LEAST]{8 16 32 64}_MAX  
INT{MAX_PTR}_MIN  
[U]INT{MAX_PTR}_MAX  
{PTRDIFF SIG_ATOMIC_WCHAR_WINT}{_MAX _MIN}  
SIZE_MAX
```

plus function macros of the form:

```
[U]INT{8 16 32 64 MAX}_C
```

-2- The header defines all types and macros the same as the C standard library header `<stdint.h>`. SEE ALSO: ISO C 7.20

-?- In particular, all types that use the placeholder *N* are optional when *N* is not 8, 16, 32 or 64. The exact-width types `intN t` and `uintN t` for *N* = 8, 16, 32, 64 are also optional; however, if an implementation provides integer types with the corresponding width, no padding bits, and (for the signed types) that have a two's complement representation, it defines the corresponding typedef names. Only those macros are defined that correspond to typedef names that the implementation actually provides. [Note: The macros `INTN C` and `UINTN C` correspond to the typedef names `int_leastN t` and `uint_leastN t`, respectively. — end note]

2. Change 29.12.2 [[cinttypes.syn](#)] as indicated:

```
#define PRIidN see below  
#define PRIiN see below  
#define PRIoN see below  
#define PRIuN see below  
#define PRIdN see below  
#define PRIdN see below  
#define SCNdN see below  
#define SCNiN see below  
#define SCNoN see below  
#define SCNuN see below  
#define SCNxN see below  
#define PRIidLEASTN see below  
#define PRIiLEASTN see below  
#define PRIoLEASTN see below  
#define PRIuLEASTN see below  
#define PRIdLEASTN see below  
#define PRIdLEASTN see below  
#define SCNdLEASTN see below  
#define SCNiLEASTN see below  
#define SCNoLEASTN see below  
#define SCNuLEASTN see below  
#define SCNxLEASTN see below  
#define PRIidFASTN see below  
#define PRIiFASTN see below  
#define PRIoFASTN see below  
#define PRIuFASTN see below  
#define PRIdFASTN see below  
#define PRIdFASTN see below  
#define SCNdFASTN see below  
#define SCNiFASTN see below  
#define SCNoFASTN see below  
#define SCNuFASTN see below  
#define SCNxFASTN see below  
[...]
```

-1- The contents and meaning of the header `<cinttypes>` [...]

-?- PRI macros that use the placeholder *N* are defined if and only if the implementation actually provides the corresponding typedef name in 17.4.2 [[cstdint.syn](#)]. SCN macros that use the placeholder *N* are defined if and only if the implementation actually provides the corresponding typedef name and the implementation has a suitable `fscanf` length modifier for the type.

[2019-03-11; Reflector review and improved wording]

Wording simplifications due to new general two's complement requirements of integer types; removal of wording redundancies and applying some typo fixes in macro names.

[2019-03-16; Daniel comments and updates wording]

Hubert Tong pointed out that we do not have a statement about `[U]INTPTR_{MIN|MAX}` being optional. Interestingly, the C11 Standard does not say directly that the `[U]INTPTR_{MIN|MAX}` macros are optional, but this follows indirectly from the fact that `intptr_t` and `uintptr_t` are indeed optional. The updated wording therefore realizes Hubert's suggestion.

In addition, the reference document has been rebased to [N4810](#), because that draft version contains an [editorial change](#), which renames the term "range exponent" of integer types to "width", which is the vocabulary used below and also matches C's use.

Finally, Hubert Tong suggested the following rewording replacements of

If and only if the implementation defines such a typedef name, it also defines the corresponding macros.

to:

Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding typedef name.

and of

PRI macros that use the placeholder N are defined if and only if the implementation actually defines the corresponding typedef name in 17.4.2 [\[cstdint.syn\]](#). SCN macros that use the placeholder n are defined if and only if the implementation actually defines the corresponding typedef name and the implementation has a suitable fscanf length modifier for the type.

to:

Each of the macros listed in this subclause is defined if and only if the implementation actually defines the corresponding typedef name in 17.4.2 [\[cstdint.syn\]](#).

Those changes have been applied as well.

[2019-03-26; Reflector discussion and minor wording update]

Geoffrey pointed out that the revised wording has the effect that it requires an implementation to define SCN macros for all mentioned typedefs, but the C11 standard says "the corresponding fscanf macros shall be defined unless the implementation does not have a suitable fscanf length modifier for the type.". An additional wording update repairs this problem below.

[2020-02-22; Reflector discussion]

Status set to Tentatively Ready after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4849](#).

1. Change 17.4.2 [\[cstdint.syn\]](#), header <cstdint> synopsis, as indicated:

```
[...]
using int64_t = signed integer type; // optional
using intN_t = see below; // optional, see below
[...]
using int_fast64_t = signed integer type;
using int_fastN_t = see below; // optional, see below
[...]
using int_least64_t = signed integer type;
using int_leastN_t = see below; // optional, see below
[...]
using uint64_t = unsigned integer type; // optional
using uintN_t = see below; // optional, see below
[...]
using uint_fast64_t = unsigned integer type;
using uint_fastN_t = see below; // optional, see below
[...]
using uint_least64_t = unsigned integer type;
using uint_leastN_t = see below; // optional, see below

using uintmax_t = unsigned integer type;
using uintptr_t = unsigned integer type; // optional

#define INTN_MIN see below
#define INTN_MAX see below
#define UINTN_MAX see below

#define INT_FASTN_MIN see below
#define INT_FASTN_MAX see below
#define UINT_FASTN_MAX see below

#define INT_LEASTN_MIN see below
#define INT_LEASTN_MAX see below
#define UINT_LEASTN_MAX see below

#define INTMAX_MIN see below
#define INTMAX_MAX see below
#define UINTMAX_MAX see below

#define INTPTR_MIN optional, see below
#define INTPTR_MAX optional, see below
#define UINTPTR_MAX optional, see below

#define PTRDIFF_MIN see below
#define PTRDIFF_MAX see below
#define SIZE_MAX see below

#define SIG_ATOMIC_MIN see below
#define SIG_ATOMIC_MAX see below

#define WCHAR_MIN see below
#define WCHAR_MAX see below

#define WINT_MIN see below
#define WINT_MAX see below

#define INTN_C(value) see below
#define UINTN_C(value) see below
```

```
#define INTMAX C(value) see below
#define UINTMAX C(value) see below
```

-1- The header also defines numerous macros of the form:

```
INT_[FAST LEAST]{8 16 32 64}_MIN
[U]INT_[FAST LEAST]{8 16 32 64}_MAX
INT{MAX_PTR}_MIN
[U]INT{MAX_PTR}_MAX
{PTRDIFF SIG_ATOMIC_WCHAR_WINT}{_MAX _MIN}
SIZE_MAX
```

plus function macros of the form:

```
{U}INT{8 16 32 64 MAX}_G
```

-2- The header defines all types and macros the same as the C standard library header `<stdint.h>`. SEE ALSO: ISO C 7.20

-?- All types that use the placeholder *N* are optional when *N* is not 8, 16, 32 or 64. The exact-width types `intN t` and `uintN t` for *N* = 8, 16, 32, 64 are also optional; however, if an implementation defines integer types with the corresponding width and no padding bits, it defines the corresponding typedef names. Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding typedef name. [Note: The macros `INTN C` and `UINTN C` correspond to the typedef names `int_leastN t` and `uint_leastN t`, respectively. — end note]

2. Change 29.12.2 [cinttypes.syn] as indicated:

```
#define PRIdN see below
#define PRIiN see below
#define PRIoN see below
#define PRIuN see below
#define PRIdN see below
#define PRIiN see below
#define PRIoN see below
#define PRIuN see below
#define SCNdN see below
#define SCNiN see below
#define SCNoN see below
#define SCNuN see below
#define SCNxN see below
#define PRIdLEASTN see below
#define PRIiLEASTN see below
#define PRIoLEASTN see below
#define PRIuLEASTN see below
#define PRIdLEASTN see below
#define PRIiLEASTN see below
#define PRIoLEASTN see below
#define PRIuLEASTN see below
#define SCNdLEASTN see below
#define SCNiLEASTN see below
#define SCNoLEASTN see below
#define SCNuLEASTN see below
#define SCNxLEASTN see below
#define PRIdFASTN see below
#define PRIiFASTN see below
#define PRIoFASTN see below
#define PRIuFASTN see below
#define PRIdFASTN see below
#define PRIiFASTN see below
#define PRIoFASTN see below
#define PRIuFASTN see below
#define SCNdFASTN see below
#define SCNiFASTN see below
#define SCNoFASTN see below
#define SCNuFASTN see below
#define SCNxFASTN see below
[...]
```

-1- The contents and meaning of the header `<cinttypes>` [...]

-?- Each of the PRI macros listed in this subclause is defined if and only if the implementation defines the corresponding typedef name in 17.4.2 [cstdint.syn]. Each of the SCN macros listed in this subclause is defined if and only if the implementation defines the corresponding typedef name in 17.4.2 [cstdint.syn] and has a suitable fscanf length modifier for the type.

3036. polymorphic_allocator::destroy is extraneous

Section: 20.12.3 [mem.poly.allocator.class] Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2017-11-15 Last modified: 2020-10-11

Priority: 3

View all other [issues](#) in [mem.poly.allocator.class].

Discussion:

polymorphic_allocator's member function destroy is exactly equivalent to the default implementation of destroy in allocator_traits (20.10.9.3 [allocator.traits.members] para 6). It should be struck from polymorphic_allocator as it provides no value.

[28-Nov-2017 Mailing list discussion - set priority to P3]

PJ says that Dinkumware is shipping an implementation of polymorphic_allocator with destroy, so removing it would be a breaking change for him.

[2019-02; Kona Wednesday night issue processing]

Status to Open; revisit once [P0339](#) lands. Poll taken was 5-3-2 in favor of removal.

[2020-10-05; Jonathan provides new wording]

Previous resolution [SUPERSEDED]:

Wording relative to [N4700](#).

1. Strike the declaration of destroy from the synopsis of class polymorphic_allocator in 20.12.3 [\[mem.poly.allocator.class\]](#):

```
template <class T1, class T2, class U, class V>
    void construct(pair<T1,T2>* p, pair<U, V>&& pr);

template <class T>
    void destroy(T* p);

polymorphic_allocator select_on_container_copy_construction() const;
```

2. Strike the specification of destroy in 20.12.3.3 [\[mem.poly.allocator.mem\]](#):

```
[...]

template <class T>
    void destroy(T* p);

-14 Effects: As if by p->~T();

[...]
```

[2020-10-11; Reflector poll]

Moved to Tentatively Ready after seven votes in favour.

Proposed resolution:

Wording relative to [N4861](#).

1. Strike the declaration of destroy from the synopsis of class polymorphic_allocator in 20.12.3 [\[mem.poly.allocator.class\]](#):

```
template <class T1, class T2, class U, class V>
    void construct(pair<T1,T2>* p, pair<U, V>&& pr);

template <class T>
    void destroy(T* p);

polymorphic_allocator select_on_container_copy_construction() const;
```

2. Adjust the specification of delete_object in 20.12.3.3 [\[mem.poly.allocator.mem\]](#):

```
template <class T>
    void delete_object(T* p);

-13- Effects: Equivalent to:

    allocator_traits<polymorphic_allocator>::destroy(*this, p);
    deallocate_object(p);
```

3. Strike the specification of destroy in 20.12.3.3 [\[mem.poly.allocator.mem\]](#):

```
[...]

template <class T>
    void destroy(T* p);

-17- Effects: As if by p->~T();

[...]
```

4. Add a new subclause to Annex D:

D.?? Deprecated polymorphic allocator member function

-1- The following member is declared in addition to those members specified in 20.12.3.3 [\[mem.poly.allocator.mem\]](#):

```
namespace std::pmr {
    template<class Tp = byte>
    class polymorphic_allocator {
    public:
        template <class T>
            void destroy(T* p);
    };
};
```

```
template <class T>
void destroy(T* p);
```

-1- Effects: As if by `p->~T()`.

3120. Unclear behavior of `monotonic_buffer_resource::release()`

Section: 20.12.6.3 [[mem.res.monotonic.buffer.mem](#)] Status: [Tentatively Ready](#) Submitter: Arthur O'Dwyer Opened: 2018-06-10 Last modified: 2020-10-06

Priority: 2

View all other [issues](#) in [[mem.res.monotonic.buffer.mem](#)].

Discussion:

The effects of `monotonic_buffer_resource::release()` are defined as:

Calls `upstream_rsrc->deallocate()` as necessary to release all allocated memory.

This doesn't give any instruction on what to do with the memory controlled by the `monotonic_buffer_resource` which was not allocated, i.e., what to do with the initial buffer provided to its constructor.

Boost.Container's pmr implementation expels its initial buffer after a `release()`. Arthur O'Dwyer's proposed pmr implementation for libc++ reuses the initial buffer after a `release()`, on the assumption that this is what the average library user will be expecting.

```
#include <memory_resource>

int main()
{
    char buffer[100];
    {
        std::pmr::monotonic_buffer_resource mr(buffer, 100, std::pmr::null_memory_resource());
        mr.release();
        mr.allocate(60); // A
    }
    {
        std::pmr::monotonic_buffer_resource mr(buffer, 100, std::pmr::null_memory_resource());
        mr.allocate(60); // B
        mr.release();
        mr.allocate(60); // C
    }
}
```

Assume that allocation "B" always succeeds.

With the proposed libc++ implementation, allocations "A" and "C" both succeed.

With Boost.Container's implementation, allocations "A" and "C" both fail.

Using another plausible implementation strategy, allocation "A" could succeed but allocation "C" could fail. I have been informed that MSVC's implementation does this.

Which of these strategies should be permitted by the Standard?

Arthur considers "A and C both succeed" to be the obviously most user-friendly strategy, and really really hopes it's going to be permitted. Requiring "C" to succeed is unnecessary (and would render MSVC's current implementation non-conforming) but could help programmers concerned with portability between different implementations.

Another side-effect of `release()` which goes underspecified by the Standard is the effect of `release()` on `next_buffer_size`. As currently written, my interpretation is that `release()` is not permitted to decrease `current_buffer_size`; I'm not sure if this is a feature or a bug.

Consider this test case (taken from [here](#)):

```
std::pmr::monotonic_buffer_resource mr(std::pmr::new_delete_resource());
for (int i=0; i < 100; ++i) {
    mr.allocate(1); // D
    mr.release();
}
```

Arthur believes it is important that the 100th invocation of line "D" does not attempt to allocate 2^{100} bytes from the upstream resource.

[2018-06-23 after reflector discussion]

Priority set to 2

Previous resolution [SUPERSEDED]:

This wording is relative to [N4750](#).

[Drafting note: The resolution depicted below would make MSVC's and my-proposed-libc++'s implementations both conforming.]

1. Modify 20.12.6.3 [\[mem.res.monotonic.buffer.mem\]](#) as indicated:

```
void release();
```

-1- *Effects*: Calls `upstream_rsrc->deallocate()` as necessary to release all allocated memory. Resets the state of the initial buffer.

-2- [*Note*: The memory is released back to `upstream_rsrc` even if some blocks that were allocated from this have not been deallocated from this. This function has an unspecified effect on next buffer size. — *end note*]

[2018-08-23 Batavia Issues processing]

We liked Pablo's wording from the reflector discussion. Status to Open.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4750](#).

1. Modify 20.12.6.3 [\[mem.res.monotonic.buffer.mem\]](#) as indicated:

```
void release();
```

-1- *Effects*: Calls `upstream_rsrc->deallocate()` as necessary to release all allocated memory. Resets *this to its initial state at construction.

[2020-10-03; Daniel comments and provides improved wording]

The recent wording introduces the very generic term "state" without giving a concrete definition of that term. During reflector discussions different interpretations of that term were expressed. The revised wording below gets rid of that word and replaces it by the actually involved exposition-only members.

[2020-10-06; moved to Tentatively Ready after seven votes in favour in reflector poll]

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 20.12.6.3 [\[mem.res.monotonic.buffer.mem\]](#) as indicated:

```
void release();
```

-1- *Effects*: Calls `upstream_rsrc->deallocate()` as necessary to release all allocated memory. Resets current buffer and next buffer size to their initial values at construction.

[3170](#). `is_always_equal` added to `std::allocator` makes the standard library treat derived types as always equal

Section: 20.10.10 [\[default.allocator\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Billy O'Neal III **Opened:** 2018-11-29 **Last modified:** 2020-10-02

Priority: 2

View all other [issues](#) in [\[default.allocator\]](#).

Discussion:

I (Billy O'Neal) attempted to change MSVC++'s standard library to avoid instantiating allocators' `operator==` for allocators that are declared `is_always_equal` to reduce the number of template instantiations emitted into `.objs`.

In so doing I introduced an unrelated bug related to POCMA handling, but it brought my attention to [this allocator](#). This allocator doesn't meet the allocator requirements because it is getting `std::allocator`'s `operator==` and `operator!=` which don't compare the root member. However, if this had been a conforming C++14 allocator with its own `==` and `!=` we would still be treating it as `is_always_equal`, as it picks that up by deriving from `std::allocator`.

`std::allocator` doesn't actually need `is_always_equal` because the defaults provided by `allocator_traits` will say `true_type` for it, since implementers don't make `std::allocator` stateful.

Billy O'Neal thinks this is NAD on the grounds that we need to be able to add things or change the behavior of standard library types.

Stephan T Lavavej thinks we should resolve this anyway because we don't know of an implementation for which this would change the default answer provided by `allocator_traits`.

[2019-02 Priority set to 2 after reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4778](#).

1. Modify 20.10.10 [\[default.allocator\]](#) as follows:

-1- All specializations of the default allocator satisfy the allocator completeness requirements (16.4.4.6.2 [\[allocator.requirements.completeness\]](#)).

```
namespace std {
    template<class T> class allocator {
    public:
        using value_type = T;
        using size_type = size_t;
        using difference_type = ptrdiff_t;
        using propagate_on_container_move_assignment = true_type;
        using is_always_equal = true_type;
        constexpr allocator() noexcept;
        constexpr allocator(const allocator&) noexcept;
        template<class U> constexpr allocator(const allocator<U>&) noexcept;
        ~allocator();
        allocator& operator=(const allocator&) = default;
        [[nodiscard]] T* allocate(size_t n);
        void deallocate(T* p, size_t n);
    };
}
```

~~?- allocator traits<allocator<T>>::is_always_equal::value is true for any T.~~

[2019-07 Cologne]

Jonathan provides updated wording.

[2020-10-02; Issue processing telecon: Moved to Tentatively Ready.]

Proposed resolution:

This wording is relative to [N4820](#).

1. Modify 20.10.10 [\[default.allocator\]](#) as follows:

-1- All specializations of the default allocator satisfy the allocator completeness requirements (16.4.4.6.2 [\[allocator.requirements.completeness\]](#)).

```
namespace std {
    template<class T> class allocator {
    public:
        using value_type = T;
        using size_type = size_t;
        using difference_type = ptrdiff_t;
        using propagate_on_container_move_assignment = true_type;
        using is_always_equal = true_type;
        constexpr allocator() noexcept;
        constexpr allocator(const allocator&) noexcept;
        template<class U> constexpr allocator(const allocator<U>&) noexcept;
        ~allocator();
        allocator& operator=(const allocator&) = default;
        [[nodiscard]] T* allocate(size_t n);
        void deallocate(T* p, size_t n);
    };
}
```

~~?- allocator traits<allocator<T>>::is_always_equal::value is true for any T.~~

2. Add a new subclause in Annex D after D.13 [\[depr.str.streams\]](#):

D.? The default allocator [\[depr.default.allocator\]](#)

~~?- The following member is defined in addition to those specified in 20.10.10 [\[default.allocator\]](#):~~

```
namespace std {
    template <class T> class allocator {
    public:
        using is_always_equal = true_type;
    };
}
```

3171. LWG 2989 breaks directory_entry stream insertion

Section: 29.11.10 [\[fs.class.directory.entry\]](#) Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2018-12-03 Last modified: 2020-08-21

Priority: 2

View other [active issues](#) in [\[fs.class.directory.entry\]](#).

View all other [issues](#) in [\[fs.class.directory.entry\]](#).

Discussion:

directory_entry has a conversion function to const path& and depends on path's stream insertion operator for stream insertion support, which is now broken after LWG 2989 made it a hidden friend.

This does not appear to be intended.

[2018-12-21 Reflector prioritization]

Set Priority to 2

[2019-02; Kona Wednesday night issue processing]

Status to Open; Marshall to move definition inline and re-vote on reflector.

Jonathan to write a paper about how to specify "hidden friends".

Previous resolution [SUPERSEDED]:

This wording is relative to [N4778](#).

1. Modify [fs.class.directory_entry], class directory_entry synopsis, as follows:

```
namespace std::filesystem {
    class directory_entry {
    public:
        [...]
    private:
        filesystem::path pathobject;    // exposition only
        friend class directory_iterator; // exposition only

        template<class charT, class traits>
        friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const directory_entry& d);
    };
}
```

2. Add a new subclause at the end of [fs.class.directory_entry], as follows:

28.11.11.4 Inserter [fs.dir.entry.io]

```
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const directory_entry& d);

-1- Effects: Equivalent to: return os << d.path();
```

[2020-05-02; Daniel resyncs wording with recent working draft and comments]

We have now the paper [P1965R0](#), which introduced a specification of what friend functions in the library specification (see 16.4.6.6 [\[hidden.friends\]](#)) are supposed to mean, there is no longer an inline definition needed to clarify the meaning. In addition to updating the change of section names the provided wording has moved the friend declaration into the public part of the class definition as have done in all other cases where we take advantage of "hidden friends" declarations.

[2020-08-21 Issue processing telecon: moved to Tentatively Ready]

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 29.11.10 [\[fs.class.directory_entry\]](#), class directory_entry synopsis, as follows:

```
namespace std::filesystem {
    class directory_entry {
    public:
        [...]
        bool operator==(const directory_entry& rhs) const noexcept;
        strong_ordering operator<=>(const directory_entry& rhs) const noexcept;

        // 29.11.11.? [fs.dir.entry.io], inserter
        template<class charT, class traits>
        friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const directory_entry& d);
    private:
        [...]
    };
}
```

2. Add a new subclause at the end of 29.11.10 [\[fs.class.directory_entry\]](#), as indicated:

29.11.11.? Inserter [fs.dir.entry.io]

```
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const directory_entry& d);
```

3306. `ranges::advance` violates its preconditions

Section: 23.4.4.2 [\[range.iter.op.advance\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2019-10-27 **Last modified:** 2020-08-21

Priority: 2

View other [active issues](#) in [\[range.iter.op.advance\]](#).

View all other [issues](#) in [\[range.iter.op.advance\]](#).

Discussion:

Recall that "[*i*, *s*) denotes a range" for an iterator *i* and sentinel *s* means that either *i* == *s* holds, or *i* is dereferenceable and [*++i*, *s*) denotes a range ([\[iterator.requirements.general\]](#)).

The three-argument overload `ranges::advance(i, n, bound)` is specified in 23.4.4.2 [\[range.iter.op.advance\]](#) paragraphs 5 through 7. Para 5 establishes a precondition that [*bound*, *i*) denotes a range when *n* < 0 (both *bound* and *i* must have the same type in this case). When `sized_sentinel_for<S, I>` holds and *n* < *bound* - *i*, para 6.1.1 says that `ranges::advance(i, n, bound)` is equivalent to `ranges::advance(i, bound)`. Para 3, however, establishes a precondition for `ranges::advance(i, bound)` that [*i*, *bound*) denotes a range. [*bound*, *i*) and [*i*, *bound*) cannot both denote ranges unless *i* == *bound*, which is not the case for all calls that reach 6.1.1.

The call in para 6.1.1 wants the effects of either 4.1 - which really has no preconditions - or 4.2, which is well-defined if either [*i*, *bound*) or [*bound*, *i*) denotes a range. Para 3's stronger precondition is actually only required by Para 4.3, which increments *i* blindly looking for *bound*. The straight-forward fix here seems to be to relax para 3's precondition to only apply when 4.3 will be reached.

[2019-11 Priority to 2 during Monday issue prioritization in Belfast]

[2020-08-21 Issue processing telecon: moved to Tentatively Ready]

Proposed resolution:

This wording is relative to [N4835](#).

1. Modify 23.4.4.2 [\[range.iter.op.advance\]](#) as indicated:

```
template<input_or_output_iterator I, sentinel_for<I> S>
constexpr void ranges::advance(I& i, S bound);
```

-3- *Expects:* `Either assignable_from<I&, S> || sized_sentinel_for<S, I> is modeled, or` [*i*, *bound*) denotes a range.

-4- *Effects:*

(4.1) — If *I* and *S* model `assignable_from<I&, S>`, equivalent to `i = std::move(bound)`.

(4.2) — Otherwise, if *S* and *I* model `sized_sentinel_for<S, I>`, equivalent to `ranges::advance(i, bound - i)`.

(4.3) — Otherwise, while `bool(i != bound)` is true, increments *i*.

3368. Exactly when does `size` return `end - begin`?

Section: 24.3.10 [\[range.prim.size\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2020-01-07 **Last modified:** 2020-09-06

Priority: 0

Discussion:

The specification of `ranges::size` in 24.3.10 [\[range.prim.size\]](#) suggests that bullet 1.3 ("Otherwise, *make-unsigned-like*(`ranges::end(E) - ranges::begin(E)`) ...") only applies when `disable_sized_range<remove_cv_t<T>>` is true. This is not the design intent, but the result of an erroneous attempt to factor out the common "`disable_sized_range` is false" requirement from the member and non-member size cases in bullets 1.2.1 and 1.2.2 that occurred between [P0896R3](#) and [P0896R4](#). The intended design has always been that a range with member or non-member size with the same syntax but different semantics may opt-out of being sized by specializing `disable_sized_range`. It has never been intended that arrays or ranges whose iterator and sentinel model `sized_sentinel_for` be able to opt out of being sized via `disable_sized_range`. `disable_sized_sentinel_for` can/must be used to opt out in the latter case so that library functions oblivious to the range type that operate on the iterator and sentinel of such a range will avoid subtraction.

[2020-01-25 Status set to Tentatively Ready after six positive votes on the reflector.]

Proposed resolution:

This wording is relative to [N4842](#).

1. Modify 24.3.10 [\[range.prim.size\]](#) as indicated:

[Drafting note: There are drive-by changes here to (1) avoid introducing unused type placeholders, (2) avoid reusing "T" as both the type of the subexpression and the template parameter of the poison pill, and (3) fix the cross-reference for *make-unsigned-like* which is

defined in [ranges.syn]/1, not in [range.subrange].]

-1- The name `size` denotes a customization point object (16.3.3.3.6 [customization.point.object]). The expression `ranges::size(E)` for some subexpression `E` with type `T` is expression-equivalent to:

(1.1) — `decay-copy(extent_v<T>)` if `T` is an array type (6.8.3 [basic.compound]).

~~(1.2) — Otherwise, if `disable_sized_range<remove_cv_t<T>>` (24.4.3 [range.sized]) is false:~~

~~(1.2.1) — Otherwise, if `disable_sized_range<remove_cv_t<T>>` (24.4.3 [range.sized]) is false and `decay-copy(E.size())` if it is a valid expression and its type `I` is of integer-like type (23.3.4.4 [iterator.concept.winc]), `decay-copy(E.size())`.~~

~~(1.2.2) — Otherwise, if `disable_sized_range<remove_cv_t<T>>` is false and `decay-copy(size(E))` if it is a valid expression and its type `I` is of integer-like type with overload resolution performed in a context that includes the declaration:~~

~~`template<class T> void size(T&&) = delete;`~~

and does not include a declaration of `ranges::size`, `decay-copy(size(E))`.

(1.3) — Otherwise, `make_unsigned_like(ranges::end(E) - ranges::begin(E))` (24.5.4 [range.subrange]24.2 [ranges.syn]) if it is a valid expression and the types `I` and `S` of `ranges::begin(E)` and `ranges::end(E)` (respectively) model both `sized_sentinel_for<S, I>` (23.3.4.8 [iterator.concept.sizedsentinel]) and `forward_iterator<I>`. However, `E` is evaluated only once.

(1.4) — [...]

3403. Domain of `ranges::ssize(E)` doesn't match `ranges::size(E)`

Section: 24.3.11 [range.prim.ssize] Status: [Tentatively Ready](#) Submitter: Jonathan Wakely Opened: 2020-02-19 Last modified: 2020-09-06

Priority: 2

Discussion:

`ranges::size(E)` works with a non-range for which `E.size()` or `size(E)` is valid. But `ranges::ssize(E)` requires the type `range_difference_t` which requires `ranges::begin(E)` to be valid. This means there are types for which `ranges::size(E)` is valid but `ranges::ssize(E)` is not.

Casey's reaction to this is:

I believe we want `ranges::ssize` to work with any argument that `ranges::size` accepts. That suggest to me that we're going to need `make_signed_like_t<T>` after all, so we can "Let `E` be an expression, and let `D` be the wider of `ptrdiff_t` or `decltype(ranges::size(E))`. Then `ranges::ssize(E)` is expression-equivalent to `static_cast<make_signed_like_t<D>>(ranges::size(E))`." Although this wording is still slightly icky since `D` isn't a valid type when `ranges::size(E)` isn't a valid expression, I think it's an improvement?

[2020-03-11 Issue Prioritization]

Priority to 2 after reflector discussion.

[2020-07-22 Casey provides wording]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

1. Add a new paragraph after paragraph 1 in 24.2 [ranges.syn]:

~~?- Also within this clause, `make_signed_like_t<X>` for an integer-like type `X` denotes `make_signed_t<X>` if `X` is an integer type; otherwise, it denotes a corresponding unspecified signed-integer-like type of the same width as `X`.~~

2. Modify 24.3.11 [range.prim.ssize] as indicated:

~~-1- The name `ranges::ssize` denotes a customization point object (16.3.3.3.6 [customization.point.object]). The expression `ranges::ssize(E)` for a subexpression `E` of type `T` is expression-equivalent to:~~

~~(1.1) — If `range_difference_t<T>` has width less than `ptrdiff_t`, `static_cast<ptrdiff_t>(ranges::size(E))`;~~

~~(1.2) — Otherwise, `static_cast<range_difference_t<T>>(ranges::size(E))`;~~

~~?- Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. If `ranges::size(t)` is ill-formed, `ranges::ssize(E)` is ill-formed. Otherwise, let `D` be the wider of `ptrdiff_t` or `decltype(ranges::size(t))`; `ranges::ssize(E)` is expression-equivalent to `static_cast<make_signed_like_t<D>>(ranges::size(t))`.~~

[2020-07-31 Casey provides updated wording]

Per discussion on the reflector.

[2020-08-21; Issue processing telecon: Tentatively Ready]

Proposed resolution:

This wording is relative to [N4861](#).

1. Add a new paragraph after paragraph 1 in 24.2 [\[ranges.syn\]](#):

[Drafting note: The following does not define an analog *to-signed-like* for *to-unsigned-like* since we don't need it at this time.]

~~[-?- Also within this Clause, *make-signed-like-t<x>* for an integer-like type *x* denotes *make_signed_t<x>* if *x* is an integer type; otherwise, it denotes a corresponding unspecified signed-integer-like type of the same width as *x*.~~

2. Modify 24.3.11 [\[range.prim ssize\]](#) as indicated:

~~-1- The name `ranges::ssize` denotes a customization point object (16.3.3.3.6 [\[customization.point.object\]](#)). The expression `ranges::ssize(E)`, for a subexpression *E* of type *T* is expression equivalent to:~~

~~(1.1) If `range_difference_t<T>` has width less than `ptrdiff_t`, `static_cast<ptrdiff_t>(ranges::size(E))`;~~

~~(1.2) Otherwise, `static_cast<range_difference_t<T>>(ranges::size(E))`;~~

~~[-?- Given a subexpression *E* with type *T*, let *t* be an lvalue that denotes the reified object for *E*. If `ranges::size(t)` is ill-formed, `ranges::ssize(E)` is ill-formed. Otherwise let *D* be *make-signed-like-t<decltype(ranges::size(t))>*, or `ptrdiff_t` if it is wider than that type; `ranges::ssize(E)` is expression-equivalent to `static_cast<D>(ranges::size(t))`.]~~

3404. Finish removing subrange's conversions from *pair-like*

Section: 24.5.4 [\[range.subrange\]](#) Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2020-02-20 Last modified: 2020-09-06

Priority: 0

View other [active issues](#) in [\[range.subrange\]](#).

View all other [issues](#) in [\[range.subrange\]](#).

Discussion:

Both LWG [3281](#) "Conversion from *pair-like* types to subrange is a silent semantic promotion" and LWG [3282](#) "subrange converting constructor should disallow derived to base conversions" removed subrange's hated implicit conversions from *pair-like* types. Notably, neither issue removed the two "*iterator-sentinel-pair*" deduction guides which target the removed constructors nor the exposition-only *iterator-sentinel-pair* concept itself, all of which are now useless.

[2020-03-11 Issue Prioritization]

Status set to Tentatively Ready after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 24.5.4 [\[range.subrange\]](#) as indicated:

```
[...]
template<class T, class U, class V>
    concept pair-like-convertible-from = // exposition only
        !range<T> && pair-like<T> && constructible_from<T, U, V>;

template<class T>
    concept iterator_sentinel_pair = // exposition only
        !range<T> && pair-like<T> &&
        sentinel_for<tuple_element_t<1, T>, tuple_element_t<0, T>>;

[...]

template<iterator_sentinel_pair P>
    subrange(P) -> subrange<tuple_element_t<0, P>, tuple_element_t<1, P>>;

template<iterator_sentinel_pair P>
    subrange(P, make_unsigned_like_t<iter_difference_t<tuple_element_t<0, P>>> ->
        subrange<tuple_element_t<0, P>, tuple_element_t<1, P>, subrange_kind::sized>;

[...]
```

3405. common_view's converting constructor is bad, too

Section: 24.7.14.2 [\[range.common.view\]](#) Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2020-02-20 Last modified: 2020-09-06

Priority: 0

Discussion:

LWG 3280 struck the problematic/extraneous converting constructor templates from the *meow_view* range adaptor types in the standard library with the exception of *common_view*. The omission of *common_view* seems to have been simply an oversight: its converting constructor template is no less problematic or extraneous. We should remove *common_view*'s converting constructor template as well to finish the task. Both cmcstl2 and range-v3 removed the converting constructor template from *common_view* when removing the other converting constructor templates, so we have implementation experience that this change is good as well as consistent with the general thrust of LWG 3280.

[2020-03-11 Issue Prioritization]

Status set to Tentatively Ready after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 24.7.14.2 [\[range.common.view\]](#), class template *common_view* synopsis, as indicated:

```
[...]
constexpr explicit common_view(V r);

template<viewable_range R>
requires (!common_range<R> && constructible_from<V, all_view<R>>)
constexpr explicit common_view(R&& r);

constexpr V base() const& requires copy_constructible<V> { return base_; }
[...]
```

```
[...]
template<viewable_range R>
requires (!common_range<R> && constructible_from<V, all_view<R>>)
constexpr explicit common_view(R&& r);

-2- Effects: Initializes base_ with views::all(std::forward<R>(r)).
```

3406. elements_view::begin() and elements_view::end() have incompatible constraints

Section: 24.7.16.2 [\[range.elements.view\]](#) Status: [Tentatively Ready](#) Submitter: Patrick Palka Opened: 2020-02-21 Last modified: 2020-10-02

Priority: 1

View all other [issues](#) in [\[range.elements.view\]](#).

Discussion:

[P1994R1](#) (*elements_view* needs its own sentinel) introduces a distinct *sentinel* type for *elements_view*. In doing so, it replaces the two existing overloads of *elements_view::end()* with four new ones:

```
constexpr auto end() requires (!simple_view<V>)
{ return ranges::end(base_); }
constexpr auto end() const requires simple_view<V>
{ return ranges::end(base_); }
+ constexpr auto end()
+ { return sentinel<false>{ranges::end(base_)}; }
+
+ constexpr auto end() requires common_range<V>
+ { return iterator<false>{ranges::end(base_)}; }
+
+ constexpr auto end() const
+ requires range<const V>
+ { return sentinel<true>{ranges::end(base_)}; }
+
+ constexpr auto end() const
+ requires common_range<const V>
+ { return iterator<true>{ranges::end(base_)}; }
```

But now these new overloads of *elements_view::end()* have constraints that are no longer consistent with the constraints of *elements_view::begin()*:

```
constexpr auto begin() requires (!simple-view<V>)
{ return iterator<false>(ranges::begin(base_)); }

constexpr auto begin() const requires simple-view<V>
{ return iterator<true>(ranges::begin(base_)); }
```

This inconsistency means that we can easily come up with a view *v* for which *elements_view<V>::begin()* returns an *iterator<true>* and *elements_view<V>::end()* returns an *sentinel<false>*, i.e. incomparable things of opposite constness. For example:

```
tuple<int, int> x[] = {{0,0}};
ranges::subrange r = {counted_iterator(x, 1), default_sentinel};
auto v = r | views::elements<0>;
v.begin() == v.end(); // ill-formed
```

Here, overload resolution for `begin()` selects the `const` overload because the subrange `r` models *simple-view*. But overload resolution for `end()` selects the non-`const` non-common_range overload. Hence the last line of this snippet is ill-formed because it is comparing an iterator and sentinel of opposite constness, for which we have no matching `operator==` overload. So in this example `v` does not even model range because its `begin()` and `end()` are incomparable.

This issue can be resolved by making sure the constraints on `elements_view::begin()` and on `elements_view::end()` are consistent and compatible. The following proposed resolution seems to be one way to achieve that and takes inspiration from the design of `transform_view`.

[2020-04-04 Issue Prioritization]

Priority to 1 after reflector discussion.

[2020-07-17; telecon]

Should be considered together with [3448](#) and [3449](#).

Previous resolution [SUPERSEDED]:

This wording is relative to [N4849](#) after application of [P1994R1](#).

1. Modify 24.7.16.2 [[range.elements.view](#)], class template `elements_view` synopsis, as indicated:

```
namespace std::ranges {
    [...]
    template<input_range V, size_t N>
        requires view<V> && has-tuple-element<range_value_t<V>, N> &&
            has-tuple-element<remove_reference_t<range_reference_t<V>>, N>
    class elements_view : public view_interface<elements_view<V, N>> {
    public:
        [...]
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() requires (!simple-view<V>)
        { return iterator<false>(ranges::begin(base_)); }
        constexpr auto begin() const requires simple-view<V> range<const V>
        { return iterator<true>(ranges::begin(base_)); }
        [...]
    };
}
```

[2020-06-05 Tim updates P/R in light of reflector discussions and LWG [3448](#) and comments]

The fact that, as currently specified, `sentinel<false>` is not comparable with `iterator<true>` is a problem with the specification of this comparison, as noted in LWG [3448](#). The P/R below repairs this problem along the lines suggested in that issue. The constraint mismatch does make this problem easier to observe for `elements_view`, but the mismatch is not independently a problem: since `begin` can only add constness on *simple-views* for which constness is immaterial, whether `end` also adds constness or not ought not to matter.

However, there is a problem with the `begin` overload set: if `const V` is a range, but `V` is not a *simple-view*, then a `const elements_view<V, N>` has no viable `begin` at all (the simplest example of such non-simple-views is probably `single_view`). That's simply broken; the fix is to constrain the `const` overload of `begin` with just `range<const V>` instead of *simple-view<V>*. Notably, this is how many other uses of *simple-view* work already (see, e.g., `take_view` in 24.7.7.2 [[range.take.view](#)]).

The previous *simple-view* constraint on `end` served a useful purpose (when done correctly): it reduces template instantiations if the underlying view is const-agnostic. This was lost in P1994 because that paper modeled the overload set on `transform_view`; however, discussion with Eric Niebler confirmed that the reason `transform_view` doesn't have the *simple-view* optimization is because it would add constness to the callable object as well, which can make a material difference in the result. Such concerns are not present in `elements_view` where the "callable object" is effectively hard-coded into the type and unaffected by const-qualification. The revised P/R below therefore restores the *simple-view* optimization.

I have implemented this P/R together with P1994 on top of libstdc++ trunk and can confirm that no existing test cases were broken by these changes, and that it fixes the issue reported here as well as in LWG [3448](#) (as it relates to `elements_view`).

[2020-10-02; Status to Tentatively Ready after five positive votes on the reflector]

Proposed resolution:

This wording is relative to [N4861](#). It assumes the *maybe-const* helper introduced by the P/R of LWG [3448](#).

1. Modify 24.7.16.2 [[range.elements.view](#)], class template `elements_view` synopsis, as indicated:

```
namespace std::ranges {
    [...]
    template<input_range V, size_t N>
        requires view<V> && has-tuple-element<range_value_t<V>, N> &&
            has-tuple-element<remove_reference_t<range_reference_t<V>>, N>
    class elements_view : public view_interface<elements_view<V, N>> {
    public:
        [...]
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() requires (!simple-view<V>)
        { return iterator<false>(ranges::begin(base_)); }

        constexpr auto begin() const requires simple-view<V> range<const V>
        { return iterator<true>(ranges::begin(base_)); }
    };
}
```

```
constexpr auto end() requires (!simple-view<V> && !common_range<V>),
{ return sentinel<false>{ranges::end(base_)}; }

constexpr auto end() requires (!simple-view<V> && common_range<V>),
{ return iterator<false>{ranges::end(base_)}; }

constexpr auto end() const requires range<const V>
{ return sentinel<true>{ranges::end(base_)}; }

constexpr auto end() const requires common_range<const V>
{ return iterator<true>{ranges::end(base_)}; }
[...];
};
}
```

2. Modify 24.7.16.4 [range.elements.sentinel] as indicated:

[Drafting note: Unlike the current P/R of LWG 3448, this P/R also changes the return type of operator- to depend on the constness of iterator rather than that of the sentinel. This is consistent with sized_sentinel_for<S, I> (23.3.4.8 [iterator.concept.sizedsentinel]), which requires decltype(i - s) to be iter_difference_t<I>.]

```
namespace std::ranges {
    template<input_range V, size_t N>
        requires view<V> && has_tuple_element<range_value_t<V>, N> &&
            has_tuple_element<remove_reference_t<range_reference_t<V>>, N>
        template<bool Const>
        class elements_view<V, F::sentinel {
            [...]
            constexpr sentinel_t<Base> base() const;

            template<bool OtherConst>
                requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
            friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

            template<bool OtherConst>
                requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
            friend constexpr range_difference_t<Base maybe_const<OtherConst, V>>
                operator-(const iterator<OtherConst>& x, const sentinel& y)
                requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

            template<bool OtherConst>
                requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
            friend constexpr range_difference_t<Base maybe_const<OtherConst, V>>
                operator-(const sentinel& x, const iterator<OtherConst>& y)
                requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
        };
    [...]

    template<bool OtherConst>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
    friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

    -4- Effects: Equivalent to: return x.current_ == y.end_;

    template<bool OtherConst>
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
    friend constexpr range_difference_t<Base maybe_const<OtherConst, V>>
        operator-(const iterator<OtherConst>& x, const sentinel& y)
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

    -5- Effects: Equivalent to: return x.current_ - y.end_;

    template<bool OtherConst>
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
    friend constexpr range_difference_t<Base maybe_const<OtherConst, V>>
        operator-(const sentinel& x, const iterator<OtherConst>& y)
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

    -6- Effects: Equivalent to: return x.end_ - y.current_;
}
```

3413. [fund.ts.v3] propagate_const's swap's noexcept specification needs to be constrained and use a trait

Section: 3.2.2.7 [fund.ts.v3::propagate_const.modifiers], 3.2.2.9 [fund.ts.v3::propagate_const.algorithms] **Status:** [Tentatively Ready](#) **Submitter:** Thomas Köppe
Opened: 2020-02-29 **Last modified:** 2020-09-06

Priority: 0

Discussion:

Addresses: fund.ts.v3

In the Fundamentals TS, the noexcept specifications of both the member and non-member swap functions for propagate_const are using the old, ill-formed pattern of attempting to use "noexcept(swap(...))" as the boolean predicate. According to LWG [2456](#), this is ill-formed, and a resolution such as in [P0185R1](#) is required.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4840](#).

1. Modify [fund.ts.v3::propagate_const.syn], header <experimental/propagate_const> synopsis, as indicated:

```
// 3.2.2.9 [fund.ts.v3::propagate_const.algorithms], propagate_const specialized algorithms
template <class T>
constexpr void swap(propagate_const<T>& pt, propagate_const<T>& pt2) noexcept(see belowis nothrow swappable v<T>);
```

2. Modify 3.2.2.1 [fund.ts.v3::propagate_const.overview], class template propagate_const synopsis, as indicated:

```
// 3.2.2.7 [fund.ts.v3::propagate_const.modifiers], propagate_const modifiers
constexpr void swap(propagate_const& pt) noexcept(see belowis nothrow swappable v<T>);
```

3. Modify 3.2.2.7 [fund.ts.v3::propagate_const.modifiers] as indicated:

```
constexpr void swap(propagate_const& pt) noexcept(see belowis nothrow swappable v<T>);
```

~~-2- The constant expression in the exception specification is noexcept(swap(t_, pt.t_)).~~

-3- Effects: swap(t_, pt.t_).

4. Modify 3.2.2.9 [fund.ts.v3::propagate_const.algorithms] as indicated:

```
template <class T>
constexpr void swap(propagate_const<T>& pt1, propagate_const<T>& pt2) noexcept(see belowis nothrow swappable v<T>);
```

~~-2- The constant expression in the exception specification is noexcept(pt1.swap(pt2)).~~

-3- Effects: pt1.swap(pt2).

[2020-03-30; Reflector discussion]

This issue has very much overlap with LWG [2561](#), especially now that the library fundamentals has been rebased to C++20 the there reported problem for the corresponding swap problem for optional is now moot. During the reflector discussion of this issue here it was also observed that the free swap template for propagate_const needs to be constrained. This has been done below in the revised wording which also attempts to use a similar style as the IS.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4840](#).

1. Modify 3.2.2.1 [fund.ts.v3::propagate_const.overview], class template propagate_const synopsis, as indicated:

```
// 3.2.2.7 [fund.ts.v3::propagate_const.modifiers], propagate_const modifiers
constexpr void swap(propagate_const& pt) noexcept(see belowis nothrow swappable v<T>);
```

2. Modify 3.2.2.7 [fund.ts.v3::propagate_const.modifiers] as indicated:

```
constexpr void swap(propagate_const& pt) noexcept(see belowis nothrow swappable v<T>);
```

~~-2- The constant expression in the exception specification is noexcept(swap(t_, pt.t_)).~~

-3- Effects: swap(t_, pt.t_).

3. Modify 3.2.2.9 [fund.ts.v3::propagate_const.algorithms] as indicated:

```
template <class T>
constexpr void swap(propagate_const<T>& pt1, propagate_const<T>& pt2) noexcept(see below);
```

~~-?- Constraints: is swappable v<T> is true.~~

~~-2- The constant expression in the exception specification is noexcept(pt1.swap(pt2)).~~

-3- Effects: pt1.swap(pt2).

~~-?- Remarks: The expression inside noexcept is equivalent to:~~

~~noexcept(pt1.swap(pt2)).~~

[2020-04-06; Wording update upon reflector discussions]

[2020-05-03 Issue Prioritization]

Status set to Tentatively Ready after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4840](#).

1. Modify 3.2.2.1 [fund.ts.v3::propagate_const.overview], class template propagate_const synopsis, as indicated:

```
// 3.2.2.7 [fund.ts.v3::propagate_const.modifiers], propagate_const modifiers
constexpr void swap(propagate_const& pt) noexcept(see belowis nothrow swappable v<T>);
```

2. Modify 3.2.2.7 [fund.ts.v3::propagate_const.modifiers] as indicated:

```
constexpr void swap(propagate_const& pt) noexcept(see belowis nothrow swappable v<T>);
```

~~2- Preconditions: Lvalues of type T are swappable (C++17 §20.5.3.2).~~

~~2- The constant-expression in the exception-specification is noexcept(swap(t_, pt.t_)).~~

~~3- Effects: swap(t_, pt.t_).~~

3. Modify 3.2.2.9 [fund.ts.v3::propagate_const.algorithms] as indicated:

```
template <class T>
constexpr void swap(propagate_const<T>& pt1, propagate_const<T>& pt2) noexcept(see below);
```

~~2- Constraints: is swappable v<T> is true.~~

~~2- The constant-expression in the exception-specification is noexcept(pt1.swap(pt2)).~~

~~3- Effects: Equivalent to: pt1.swap(pt2).~~

~~2- Remarks: The expression inside noexcept is equivalent to:~~

~~noexcept(pt1.swap(pt2)).~~

3414. [networking.ts] service_already_exists has no usable constructors

Section: 13.7 [networking.ts::async.exec.ctx] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2020-03-17 **Last modified:** 2020-09-06

Priority: 0

Discussion:

Addresses: networking.ts

In the Networking TS, the service_already_exists exception type has no constructors declared. The logic_error base class is not default constructible, so service_already_exists's implicit default constructor is defined as deleted.

Implementations can add one or more private constructors that can be used by make_service, but there seems to be little benefit to that. The Boost.Asio type of the same name has a public default constructor.

[2020-04-18 Issue Prioritization]

Status set to Tentatively Ready after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4734](#).

1. Modify 13.7 [networking.ts::async.exec.ctx] p1, as indicated:

```
// service access
template<class Service> typename Service::key_type&
use_service(execution_context& ctx);
template<class Service, class... Args> Service&
make_service(execution_context& ctx, Args&&... args);
template<class Service> bool has_service(const execution_context& ctx) noexcept;
class service_already_exists : public logic_error {
public:
    service_already_exists();
};
```

2. Add a new subclause after [async.exec.ctx.globals]:

13.7.6 Class service_already_exists [async.exec.ctx.except]

-1- The class service_already_exists defines the type of objects thrown as exceptions to report an attempt to add an existing service to an execution_context.

```
service_already_exists();
```

-2- Postconditions: what() returns an implementation-defined NTBS.

3419. §[algorithms.requirements]/15 doesn't reserve as many rights as it intends to

Section: 25.2 [algorithms.requirements] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2020-03-24 **Last modified:** 2020-09-06

Priority: 0

View other [active issues](#) in [algorithms.requirements].

View all other [issues](#) in [algorithms.requirements].

Discussion:

25.2 [algorithms.requirements]/15 says:

"The number and order of deducible template parameters for algorithm declarations are unspecified, except where explicitly stated otherwise. [Note: Consequently, the algorithms may not be called with explicitly-specified template argument lists. — end note]"

But the note doesn't follow from the normative rule. For example, we felt the need to explicitly allow deduction for `min`'s template parameter:

```
template<typename T> const T& min(const T&, const T&);
```

... but if only the order and number of *deducible* template parameters is permitted to vary, then because of the required deduction behavior of this function template, there are only three possible valid declarations:

```
template<typename T> ??? min(const T&, const T&);  
template<typename T, typename U> ??? min(const T&, const U&);  
template<typename T, typename U> ??? min(const U&, const T&);
```

(up to minor differences in the parameter type). This doesn't prohibit calls with an explicitly-specified template argument list, contrary to the claim in the note. (Indeed, because a call such as `min(1, {})` is valid, either the first of the above three overloads must be present or there must be a default template argument `typename U = T`, which further adds to the fact that there may be valid calls with an explicitly-specified template argument list.)

Also, the "explicitly stated otherwise" cases use phrasing such as: "An invocation may explicitly specify an argument for the template parameter τ of the overloads in namespace `std`." which doesn't "specify otherwise" the normative rule, but does "specify otherwise" the claim in the note.

All this leads me to believe that [algorithms.requirements]/15 is backwards: the normative rule should be a note and the note should be the normative rule.

[2020-04-04 Issue Prioritization]

Status set to Tentatively Ready after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 25.2 [algorithms.requirements] as indicated:

```
-15- The well-formedness and behavior of a call to an algorithm with an explicitly-specified template argument list is number and order of deducible template parameters for algorithm declarations are unspecified, except where explicitly stated otherwise. [Note: Consequently, an implementation can declare an algorithm with different template parameters than those presented the algorithms may not be called with explicitly-specified template argument lists. — end note]
```

3420. `cpp17-iterator` should check that the type looks like an iterator first

Section: 23.3.2.3 [iterator.traits] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2020-02-29 **Last modified:** 2020-09-06

Priority: 0

View other [active issues](#) in [iterator.traits].

View all other [issues](#) in [iterator.traits].

Discussion:

It is common in pre-C++20 code to rely on SFINAE-friendly `iterator_traits` to rule out non-iterators in template constraints (`std::filesystem::path` is one example in the standard library).

C++20 changed `iterator_traits` to automatically detect its members in some cases, and this detection can cause constraint recursion. LWG [3244](#) tries to fix this for `path` by short-circuiting the check when the source type is `path` itself, but this isn't sufficient:

```
struct Foo  
{  
    Foo(const std::filesystem::path&);  
};  
  
static_assert(std::copyable<Foo>);
```

Here the copyability determination will ask whether a `path` can be constructed from a `Foo`, which asks whether `Foo` is an iterator, which checks whether `Foo` is copyable [...].

To reduce the risk of constraint recursion, we should change *cpp17-iterator* so that it does not ask about copyability until the type is known to resemble an iterator.

[2020-04-04 Issue Prioritization]

Status set to Tentatively Ready after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 23.3.2.3 [\[iterator.traits\]](#) as indicated:

-2- The definitions in this subclause make use of the following exposition-only concepts:

```
template<class I>
concept cpp17-iterator =
    copyable<I> && requires(I i) {
        { *i } -> can-reference;
        { ++i } -> same_as<I&>;
        { *i++ } -> can-reference;
    } && copyable<I>;
```

[...]

[3421](#). Imperfect ADL emulation for *boolean-testable*

Section: 18.5.2 [\[concept.booleantestable\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Davis Herring **Opened:** 2020-03-24 **Last modified:** 2020-09-06

Priority: 0

Discussion:

18.5.2 [\[concept.booleantestable\]](#)/4 checks for "a specialization of a class template that is a member of the same namespace as *D*", which ignores the possibility of inline namespaces.

[2020-04-18 Issue Prioritization]

Status set to Tentatively Ready after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 18.5.2 [\[concept.booleantestable\]](#) as indicated:

-4- A key parameter of a function template *D* is a function parameter of type *cv X* or reference thereto, where *X* names a specialization of a class template that ~~has the same innermost enclosing non-inline namespace as *D*~~ is a member of the same namespace as *D*, and *X* contains at least one template parameter that participates in template argument deduction. [...]

[3425](#). *condition_variable_any* fails to constrain its *Lock* parameters

Section: 32.6.5 [\[thread.condition.condvarany\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2020-04-04 **Last modified:** 2020-09-06

Priority: 0

View all other [issues](#) in [\[thread.condition.condvarany\]](#).

Discussion:

32.6.5 [\[thread.condition.condvarany\]](#)/1 says "A *Lock* type shall meet the *Cpp17BasicLockable* requirements (32.2.5.2 [\[thread.req.lockable.basic\]](#)).", which is fine, but it notably doesn't require anything to be a *Lock* type or meet the requirements of a *Lock* type. Given that every member template of *condition_variable_any* has a template parameter named *Lock*, the intent is clearly to impose a requirement on the template arguments supplied for those parameters but the use of code font for "Lock" in the definition of "Lock type" is a bit subtle to establish that connection. We should specify this more clearly.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

1. Modify 32.6.5 [\[thread.condition.condvarany\]](#) as indicated:

-1- A ~~Lock type~~ Template arguments for template parameters of member templates of *conditional variable any* named *Lock* shall meet the *Cpp17BasicLockable* requirements (32.2.5.2 [\[thread.req.lockable.basic\]](#)). [Note: All of the standard mutex types meet this requirement. If a *Lock* type other than one of the standard mutex types or a *unique_lock* wrapper for a standard mutex type is used with *condition_variable_any*, the user should ensure that any necessary synchronization is in place with respect to the predicate associated with the *condition_variable_any* instance. — end note]

[2020-04-06; Tim improves wording]

[2020-04-11 Issue Prioritization]

Status set to Tentatively Ready after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 32.6.5 [\[thread.condition.condvarany\]](#) as indicated:

-1- A `Lock` type in this subclause 32.6.5 [\[thread.condition.condvarany\]](#), template arguments for template parameters named `Lock` shall meet the `Cpp17BasicLockable` requirements (32.2.5.2 [\[thread.req.lockable.basic\]](#)). [Note: All of the standard mutex types meet this requirement. If a `Lock` type other than one of the standard mutex types or a `unique_lock` wrapper for a standard mutex type is used with `condition_variable_any`, the user should ensure that any necessary synchronization is in place with respect to the predicate associated with the `condition_variable_any` instance. — end note]

[3426](#). `operator<=>(const unique_ptr<T, D>&, nullptr_t)` can't get no satisfaction

Section: 20.11.1.6 [\[unique.ptr.special\]](#) Status: [Tentatively Ready](#) Submitter: Jonathan Wakely Opened: 2020-04-09 Last modified: 2020-09-06

Priority: 0

View all other [issues](#) in [\[unique.ptr.special\]](#).

Discussion:

The constraint for `operator<=>(const unique_ptr<T, D>&, nullptr_t)` cannot be satisfied, because `std::three_way_comparable<nullptr_t>` is false, because `nullptr <=> nullptr` is ill-formed.

We can make it work as intended by comparing `x.get()` to `pointer(nullptr)` instead of to `nullptr` directly.

[2020-04-14; Replacing the functional cast by a `static_cast` as result of reflector discussion]

[2020-04-18 Issue Prioritization]

Status set to Tentatively Ready after seven positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 20.10.2 [\[memory.syn\]](#) as indicated:

```
[...]
template<class T, class D>
bool operator>=(nullptr_t, const unique_ptr<T, D>& y);
template<class T, class D>
requires three_way_comparable_with<typename unique_ptr<T, D>::pointer, nullptr_t>
compare_three_way_result_t<typename unique_ptr<T, D>::pointer, nullptr_t>
operator<=>(const unique_ptr<T, D>& x, nullptr_t);
[...]
```

2. Modify 20.11.1.6 [\[unique.ptr.special\]](#) as indicated:

```
template<class T, class D>
requires three_way_comparable_with<typename unique_ptr<T, D>::pointer, nullptr_t>
compare_three_way_result_t<typename unique_ptr<T, D>::pointer, nullptr_t>
operator<=>(const unique_ptr<T, D>& x, nullptr_t);
```

-18- Returns: `compare_three_way()(x.get(), static_cast<typename unique_ptr<T, D>::pointer>(nullptr))`.

[3427](#). `operator<=>(const shared_ptr<T>&, nullptr_t)` definition ill-formed

Section: 20.11.3.8 [\[util.smartptr.shared.cmp\]](#) Status: [Tentatively Ready](#) Submitter: Daniel Krüger Opened: 2020-04-11 Last modified: 2020-09-06

Priority: 0

View all other [issues](#) in [\[util.smartptr.shared.cmp\]](#).

Discussion:

This is similar to LWG [3426](#): This time the definition of `operator<=>(const shared_ptr<T>&, nullptr_t)` is ill-formed, whose effects are essentially specified as calling:

```
compare_three_way()(a.get(), nullptr)
```

This call will be ill-formed by constraint-violation, because both `nullptr <=> nullptr` as well as `((T*) 0) <=> nullptr` are ill-formed.

As a short-term solution we can make it work as intended — equivalent to LWG [3426](#) — by comparing `a.get()` to `(element_type*) nullptr` instead of to `nullptr` directly.

As a long-term solution we should at least consider to deprecate the mixed relational operators as well as the mixed three-way comparison operator of all our smart-pointers with `std::nullptr_t` since the core language has eliminated relational comparisons of pointers with `std::nullptr_t` with [N3624](#) four years after they had been originally accepted by [CWG 879](#). Consequently, for C++20, the mixed three-way comparison between pointers and `nullptr` is not supported either. For this long-term solution I'm suggesting to handle this via a proposal.

[2020-05-16 Reflector discussions]

Status to Tentatively Ready and priority to 0 after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 20.11.3.8 [\[util.smartptr.shared.cmp\]](#) as indicated:

```
template<class T>
strong_ordering operator<=>(const shared_ptr<T>& a, nullptr_t) noexcept;

-5- Returns: compare_three_way()(a.get(), static cast<typename shared_ptr<T>::element_type*>(nullptr)).
```

[3428](#). `single_view`'s in place constructor should be explicit

Section: 24.6.3.2 [\[range.single.view\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2020-04-07 **Last modified:** 2020-09-06

Priority: 0

Discussion:

The `in_place_t` constructor template of `single_view` is not explicit:

```
template<class... Args>
requires constructible_from<T, Args...>
constexpr single_view(in_place_t, Args&&... args);
```

so it defines an implicit conversion from `std::in_place_t` to `single_view<T>` whenever `constructible_from<T>` is modeled, which seems unlikely to be the intent.

[2020-04-18 Issue Prioritization]

Status set to Tentatively Ready after six positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 24.6.3.2 [\[range.single.view\]](#) as indicated:

```
namespace std::ranges {
    template<copy_constructible T>
        requires is_object_v<T>
        class single_view : public view_interface<single_view<T>> {
        [...]
        public:
            [...]
            template<class... Args>
                requires constructible_from<T, Args...>
                constexpr explicit single_view(in_place_t, Args&&... args);

            [...]
        };
}

[...]
```

```
template<class... Args>
constexpr explicit single_view(in_place_t, Args&&... args);

-3- Effects: Initializes value_as if by value_{in_place, std::forward<Args>(args)...}.
```

[3434](#). `ios_base` never reclaims memory for `iarray` and `parray`

Section: 29.5.3.8 [\[ios.base.cons\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Alisdair Meredith **Opened:** 2020-04-26 **Last modified:** 2020-09-06

Priority: Not Prioritized

View all other [issues](#) in `[ios.base.cons]`.

Discussion:

According to 29.5.3.6 [\[ios.base.storage\]](#) the class `ios_base` allocates memory, represented by two exposition-only pointers, `iarray` and `parray` in response to calls to `iword` and `pword`. However, the specification for the destructor in 29.5.3.8 [\[ios.base.cons\]](#) says nothing about reclaiming any allocated memory.

[2020-07-17; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after six votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 29.5.3.8 [\[ios.base.cons\]](#) as indicated:

[Drafting note: Wording modeled on container requirements]

```
~ios_base();
```

-2- *Effects*: Calls each registered callback pair `(fn, idx)` (29.5.3.7 [\[ios.base.callback\]](#)) as `(*fn)(erase_event, *this, idx)` at such time that any `ios_base` member function called from within `fn` has well-defined results. **Then, any memory obtained is deallocated.**

[3435](#). `three_way_comparable_with<reverse_iterator<int*>, reverse_iterator<const int*>>`

Section: 23.5.3.4 [\[move.iter.cons\]](#), 23.5.1.4 [\[reverse.iter.cons\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2020-04-26 **Last modified:** 2020-09-06

Priority: 2

View other [active issues](#) in [\[move.iter.cons\]](#).

View all other [issues](#) in [\[move.iter.cons\]](#).

Discussion:

Despite that `reverse_iterator<int*>` and `reverse_iterator<const int*>` are comparable with `<=>`, `three_way_comparable_with<reverse_iterator<int*>, reverse_iterator<const int*>>` is false. This unfortunate state of affairs results from the absence of constraints on `reverse_iterator`'s converting constructor: both `convertible_to<reverse_iterator<int*>, reverse_iterator<const int*>>` and `convertible_to<reverse_iterator<const int*>, reverse_iterator<int*>>` evaluate to true, despite that `reverse_iterator<int*>`'s converting constructor template is ill-formed when instantiated for `reverse_iterator<const int*>`. This apparent bi-convertibility results in ambiguity when trying to determine `common_reference_t<const reverse_iterator<int*>&, const reverse_iterator<const int*>&>`, causing the `common_reference` requirement in `three_way_comparable_with` to fail.

I think we should correct this by constraining `reverse_iterator`'s conversion constructor (and converting assignment operator, while we're here) correctly so we can use the concept to determine when it's ok to compare specializations of `reverse_iterator` with `<=>`.

`move_iterator` has similar issues due to its similarly unconstrained conversions. We should fix both similarly.

[2020-07-17; Priority set to 2 in telecon]

[2020-07-26; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 23.5.3.4 [\[move.iter.cons\]](#) as indicated:

[Drafting note: this incorporates and supercedes the P/R of LWG [3265](#).]

```
constexpr move_iterator();
```

-1- *Effects*: ~~Constructs a move_iterator, value-initializing current. Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type Iterator.~~ **Value-initializes current.**

```
constexpr explicit move_iterator(Iterator i);
```

-2- *Effects*: ~~Constructs a move_iterator, initializes current with std::move(i).~~ **Initializes current with `std::move(i)`.**

```
template<class U> constexpr move_iterator(const move_iterator<U>& u);
```

-3- ~~Mandates: U is convertible to Iterator~~ **Constraints: is same `v<U, Iterator>` is false and `const U&` models `convertible_to<Iterator>`.**

-4- *Effects*: ~~Constructs a move_iterator, initializes current with u.currentbase().~~ **Initializes current with `u.currentbase()`.**

```
template<class U> constexpr move_iterator& operator=(const move_iterator<U>& u);
```

-5- ~~Mandates: U is convertible to Iterator~~ Constraints: is same v<U, Iterator> is false, const U& models convertible to<Iterator>, and assignable from<Iterator&, const U&> is modeled.

-6- *Effects:* Assigns u.~~current~~~~base()~~ to current.

2. Modify 23.5.1.4 [\[reverse.iter.cons\]](#) as indicated:

```
template<class U> constexpr reverse_iterator(const reverse_iterator<U>& u);
```

-?- Constraints: is same v<U, Iterator> is false and const U& models convertible to<Iterator>.

-3- *Effects:* Initializes current with u.current.

```
template<class U>
constexpr reverse_iterator&
operator=(const reverse_iterator<U>& u);
```

-?- Constraints: is same v<U, Iterator> is false, const U& models convertible to<Iterator>, and assignable from<Iterator&, const U&> is modeled.

-4- *Effects:* Assigns u.~~current~~~~base()~~ to current.

-5- *Returns:* *this.

[3437](#). `__cpp_lib_polymorphic_allocator` is in the wrong header

Section: 17.3.2 [\[version.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2020-04-29 **Last modified:** 2020-09-06

Priority: 0

View all other [issues](#) in [\[version.syn\]](#).

Discussion:

17.3.2 [\[version.syn\]](#) says that `__cpp_lib_polymorphic_allocator` is also defined in `<memory>`, but `std::polymorphic_allocator` is defined in `<memory_resource>`. This seems like an error in [P1902R1](#). The macro should be in the same header as the feature it relates to.

[2020-05-09; Reflector prioritization]

Set status to Tentatively Ready after six votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 17.3.2 [\[version.syn\]](#) as indicated:

```
[...]
#define __cpp_lib_parallel_algorithm      201603L // also in <algorithm>, <numeric>
#define __cpp_lib_polymorphic_allocator  201902L // also in <memory><memory_resource>
#define __cpp_lib_quoted_string_io       201304L // also in <iomanip>
[...]
```

[3446](#). `indirectly_readable_traits` ambiguity for types with both `value_type` and `element_type`

Section: 23.3.2.2 [\[readable.traits\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2020-05-15 **Last modified:** 2020-08-21

Priority: 2

Discussion:

Per 23.3.2.2 [\[readable.traits\]](#), `indirectly_readable_traits<T>::value_type` is the same type as `remove_cv_t<T>::value_type` if it denotes an object type, or `remove_cv_t<T>::element_type` if it denotes an object type. If both `T::value_type` and `T::element_type` denote types, `indirectly_readable_traits<T>::value_type` is ill-formed. This was perhaps not the best design, given that there are iterators in the wild (Boost's unordered containers) that define both nested types. `indirectly_readable_traits` should tolerate iterators that define both nested types consistently.

[2020-07-17; Priority set to 2 in telecon]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

1. Modify 23.3.2.2 [\[readable.traits\]](#) as indicated:

```
[...]
template<class> struct cond-value-type { }; // exposition only
```

```

template<class T>
    requires is_object_v<T>
    struct cond-value-type<T> {
        using value_type = remove_cv_t<T>;
    };

template<class> struct indirectly_readable_traits { };

[...]
```

```

template<class T>
    requires requires { typename T::value_type; }
    struct indirectly_readable_traits<T>
        : cond-value-type<typename T::value_type> { };

template<class T>
    requires requires { typename T::element_type; }
    struct indirectly_readable_traits<T>
        : cond-value-type<typename T::element_type> { };

template<class T>
    requires requires {
        typename T::element_type;
        typename T::value_type;
        requires same as<
            remove_cv_t<typename T::element_type>,
            remove_cv_t<typename T::value_type>;
        };
    struct indirectly_readable_traits<T>
        : cond-value-type<typename T::value_type> { };

[...]
```

[2020-07-23; Casey improves wording per reflector discussion]

[2020-08-21; moved to Tentatively Ready after five votes in favour in reflector poll]

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 23.3.2.2 [\[readable.traits\]](#) as indicated:

```

[...]
```

```

template<class> struct cond-value-type { }; // exposition only

template<class T>
    requires is_object_v<T>
    struct cond-value-type<T> {
        using value_type = remove_cv_t<T>;
    };

template<class T>
concept has-member-value-type = requires { typename T::value_type; }; // exposition only

template<class T>
concept has-member-element-type = requires { typename T::element_type; }; // exposition only

template<class> struct indirectly_readable_traits { };

[...]
```

```

template<class has-member-value-type T>
requires requires { typename T::value_type; }
struct indirectly_readable_traits<T>
    : cond-value-type<typename T::value_type> { };

template<class has-member-element-type T>
requires requires { typename T::element_type; }
struct indirectly_readable_traits<T>
    : cond-value-type<typename T::element_type> { };

template<class has-member-value-type T>
    requires has-member-element-type<T> &&
        same as<remove_cv_t<typename T::element_type>, remove_cv_t<typename T::value_type>>
    struct indirectly_readable_traits<T>
        : cond-value-type<typename T::value_type> { };

[...]
```

[3448](#). transform_view's sentinel<false> not comparable with iterator<true>

Section: 24.7.6.4 [\[range.transform.sentinel\]](#), 24.7.11.4 [\[range.join.sentinel\]](#) Status: [Tentatively Ready](#) Submitter: Jonathan Wakely Opened: 2020-05-26 Last modified: 2020-10-02

Priority: 1

View all other [issues](#) in [range.transform.sentinel].

Discussion:

A user reported that this doesn't compile:

```
#include <list>
#include <ranges>

std::list v{1, 2}; // works if std::vector
auto view1 = v | std::views::take(2);
auto view2 = view1 | std::views::transform([] (int i) { return i; });
bool b = std::ranges::cbegin(view2) == std::ranges::end(view2);
```

The comparison is supposed to use `operator==(iterator<Const>, sentinel<Const>)` after converting `sentinel<false>` to `sentinel<true>`. However, the `operator==` is a hidden friend so is not a candidate when comparing `iterator<true>` with `sentinel<false>`. The required conversion would only happen if we'd found the operator, but we can't find the operator until after the conversion happens.

As Patrick noted, the `join_view` sentinel has a similar problem.

The proposed wording shown below has been suggested by Casey and has been implemented and tested in GCC's libstdc++.

[2020-07-17; Priority set to 1 in telecon]

Should be considered together with [3406](#) and [3449](#).

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

1. Modify 24.2 [\[ranges.syn\]](#), header `<ranges>` synopsis, as indicated:

```
[Drafting note: The project editor is kindly asked to consider replacing editorially all of the

"using Base = conditional_t<Const, const V, V>";

occurrences by

"using Base = maybe-const<Const, V>";

]

[...
namespace std::ranges {
[...
namespace views { inline constexpr unspecified filter = unspecified; }

template<bool Const, class T>
using maybe-const = conditional_t<Const, const T, T>; // exposition-only

// 24.7.6 \[range.transform\], transform view
template<input_range V, copy_constructible F>
requires view<V> && is_object_v<F> &&
regular_invocable<F&, range_reference_t<V>>
class transform_view;
[...
}
```

2. Modify 24.7.6.4 [\[range.transform.sentinel\]](#), class template `transform_view::sentinel` synopsis, as indicated:

```
namespace std::ranges {
template<input_range V, copy_constructible F>
requires view<V> && is_object_v<F> &&
regular_invocable<F&, range_reference_t<V>> &&
can_reference_invoke_result_t<F&, range_reference_t<V>>>
template<bool Const>
class transform_view<V, F>::sentinel {
[...
constexpr sentinel_t<Base> base() const;

template<bool OtherConst>
requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator_t<OtherConst>& x, const sentinel_t<Base>& y);

template<bool OtherConst>
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<Base>
operator-(const iterator_t<OtherConst>& x, const sentinel_t<Base>& y)
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

template<bool OtherConst>
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<Base>
operator-(const sentinel_t<Base>& x, const iterator_t<OtherConst>& y)
```

```

        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
    };
}

[...]

template<bool OtherConst>
requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

-4- Effects: Equivalent to: return x.current_ == y.end_;

template<bool OtherConst>
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<Base>
operator-(const iterator<OtherConst>& x, const sentinel& y)
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

-5- Effects: Equivalent to: return x.current_ - y.end_;

template<bool OtherConst>
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<Base>
operator-(const sentinel& y, const iterator<OtherConst>& x)
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

-6- Effects: Equivalent to: return y.end_ - x.current_;

```

3. Modify 24.7.11.4 [\[range.join.sentinel\]](#), class template `join_view::sentinel` synopsis, as indicated:

```

namespace std::ranges {
    template<input_range V>
    requires view<V> && input_range<range_reference_t<V>> &&
        (is_reference_v<range_reference_t<V>> ||
         view<range_value_t<V>>)
    template<bool Const>
    class join_view<V>::sentinel {
    [...]

        template<bool OtherConst>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
        friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
    };
}

[...]

template<bool OtherConst>
requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

-3- Effects: Equivalent to: return x.outer_ == y.end_;

```

[2020-08-21 Tim updates PR per telecon discussion]

As noted in the PR of LWG [3406](#), the return type of `operator-` should be based on the constness of the iterator rather than that of the sentinel, as `sized_sentinel_for<S, I>` (23.3.4.8 [\[iterator.concept.sizedsentinel\]](#)) requires `decltype(i - s)` to be `iter_difference_t<I>`.

[2020-10-02; Status to Tentatively Ready after five positive votes on the reflector]

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 24.2 [\[ranges.syn\]](#), header `<ranges>` synopsis, as indicated:

[Drafting note: The project editor is kindly asked to consider replacing editorially all of the

```

"using Base = conditional_t<Const, const V, V>;"

```

occurrences by

```

"using Base = maybe-const<Const, V>;"

```

]

```

[...]
namespace std::ranges {
[...]
    namespace views { inline constexpr unspecified filter = unspecified; }

    template<bool Const, class T>
    using maybe-const = conditional_t<Const, const T, T>; // exposition-only

```

```
// 24.7.6 [range.transform], transform view
template<input_range V, copy_constructible F>
    requires view<V> && is_object_v<F> &&
        regular_invocable<F&, range_reference_t<V>>
class transform_view;
[...]
```

2. Modify 24.7.6.4 [range.transform.sentinel], class template transform_view::sentinel synopsis, as indicated:

```
namespace std::ranges {
    template<input_range V, copy_constructible F>
        requires view<V> && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<V>> &&
                can_reference_invoke_result_t<F&, range_reference_t<V>>>
    template<bool Const>
    class transform_view<V, F>::sentinel {
    public:
        constexpr sentinel_t<Base> base() const;

        template<bool OtherConst>
            requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
        friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

        template<bool OtherConst>
            requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
        friend constexpr range_difference_t<Base, maybe_const<OtherConst, V>>
            operator-(const iterator<OtherConst>& x, const sentinel& y)
            requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

        template<bool OtherConst>
            requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
        friend constexpr range_difference_t<Base, maybe_const<OtherConst, V>>
            operator-(const sentinel& y, const iterator<OtherConst>& x)
            requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
    };
}

[...]
```

template<bool OtherConst>
requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

-4- Effects: Equivalent to: return x.current_ == y.end_;

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
friend constexpr range_difference_t<Base, maybe_const<OtherConst, V>>
    operator-(const iterator<OtherConst>& x, const sentinel& y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

-5- Effects: Equivalent to: return x.current_ - y.end_;
```

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
friend constexpr range_difference_t<Base, maybe_const<OtherConst, V>>
    operator-(const sentinel& y, const iterator<OtherConst>& x)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

-6- Effects: Equivalent to: return y.end_ - x.current_;
```

3. Modify 24.7.11.4 [range.join.sentinel], class template join_view::sentinel synopsis, as indicated:

```
namespace std::ranges {
    template<input_range V>
        requires view<V> && input_range<range_reference_t<V>> &&
            (is_reference_v<range_reference_t<V>> ||
             view<range_value_t<V>>)
    template<bool Const>
    class join_view<V>::sentinel {
    public:
        [...]
```

template<bool OtherConst>
requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

};

[...]

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

-3- Effects: Equivalent to: return x.outer_ == y.end_;
```

Section: 24.7.7.3 [\[range.take.sentinel\]](#), 24.7.8.3 [\[range.take.while.sentinel\]](#) Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2020-06-06 Last modified: 2020-10-02

Priority: 1

Discussion:

During reflector discussion it was noted that the issue observed in LWG [3448](#) applies to `take_view::sentinel` and `take_while_view::sentinel` as well.

[2020-06-15 Tim corrects a typo in the PR]

[2020-10-02; Priority set to 1 in telecon]

Should be considered together with [3406](#) and [3448](#).

[2020-10-02; Status to Tentatively Ready after five positive votes on the reflector]

Proposed resolution:

This wording is relative to [N4861](#). It assumes the *maybe-const* helper introduced by the P/R of LWG [3448](#).

1. Modify 24.7.7.3 [\[range.take.sentinel\]](#) as indicated:

[Drafting note: Unlike LWG [3448](#), these operators can't be easily templated, so the approach taken here is to add an overload for the opposite constness instead. However, because `const V` may not even be a range (and therefore `iterator_t<const V>` may not be well-formed), we need to defer instantiating the signature of the new overload until the point of use.]

```
namespace std::ranges {
    template<view V>
    template<bool Const>
    class take_view<V>::sentinel {
    private:
        using Base = conditional_t<Const, const V, V>; // exposition only
        template<bool OtherConst>
        using CI = counted_iterator<iterator_t<Base maybe-const<OtherConst, V>>>; // exposition only

        [...]

        constexpr sentinel_t<Base> base() const;

        friend constexpr bool operator==(const CI<Const>& y, const sentinel& x);

        template<bool OtherConst = !Const>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
        friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x);
    };
}

[...]
```

```
friend constexpr bool operator==(const CI<Const>& y, const sentinel& x);

template<bool OtherConst = !Const>
requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x);
```

-4- Effects: Equivalent to: `return y.count() == 0 || y.base() == x.end_;`

2. Modify 24.7.8.3 [\[range.take.while.sentinel\]](#) as indicated:

```
namespace std::ranges {
    template<view V, class Pred>
    requires input_range<V> && is_object_v<Pred> &&
        indirect_unary_predicate<const Pred, iterator_t<V>>
    template<bool Const>
    class take_while_view<V>::sentinel {
    [...]

        constexpr sentinel_t<Base> base() const { return end_; }

        friend constexpr bool operator==(const iterator_t<Base>& x, const sentinel& y);

        template<bool OtherConst = !Const>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
        friend constexpr bool operator==(const iterator_t<maybe-const<OtherConst, V>>& x, const sentinel& y);
    };
}

[...]
```

```
friend constexpr bool operator==(const iterator_t<Base>& x, const sentinel& y);

template<bool OtherConst = !Const>
```

```
requires sentinel_for<sentinel t<Base>, iterator t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator t<maybe-const<OtherConst, V>>& x, const sentinel& y);
```

-3- *Effects*: Equivalent to: `return y.end_ == x || !invoke(*y.pred_, *x);`

3453. Generic code cannot call `ranges::advance(i, s)`

Section: 23.4.4.2 [\[range.iter.op.advance\]](#), 23.3.4.7 [\[iterator.concept.sentinel\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2020-06-18 **Last modified:** 2020-09-06

Priority: 2

View other [active issues](#) in [\[range.iter.op.advance\]](#).

View all other [issues](#) in [\[range.iter.op.advance\]](#).

Discussion:

The specification of the iterator & sentinel overload of `ranges::advance` in 23.4.4.2 [\[range.iter.op.advance\]](#) reads:

```
template<input_or_output_iterator I, sentinel_for<I> S>
constexpr void ranges::advance(I& i, S bound);
```

-3- *Preconditions*: `[i, bound)` denotes a range.

-4- *Effects*:

(4.1) — If `I` and `S` model `assignable_from<I&, S>`, equivalent to `i = std::move(bound)`.

(4.2) — [...]

The assignment optimization in bullet 4.1 is just fine for callers with concrete types who can decide whether or not to call `advance` depending on the semantics of the assignment performed. However, since this assignment operation isn't part of the `input_or_output_iterator` or `sentinel_for` requirements its semantics are unknown for arbitrary types. Effectively, generic code is forbidden to call this overload of `advance` when `assignable_from<I&, S>` is satisfied and non-generic code must tread lightly. This seems to make the library dangerously unusable. We can correct this problem by either:

- A. Making the assignment operation in question an optional part of the `sentinel_for` concept with well-defined semantics. This concept change should be relatively safe given that `assignable_from<I&, S>` requires `common_reference_with<const I&, const S&>`, which is very rarely satisfied inadvertently.
- B. Requiring instead `same_as<I, S>` to trigger the assignment optimization in bullet 4.1 above. `S` is semiregular, so `i = std::move(s)` is certainly well-formed (and has well-defined semantics thanks to semiregular) when `I` and `S` are the same type. The optimization will not apply in as many cases, but we don't need to make a scary concept change, either.

[2020-06-26; Reflector prioritization]

Set priority to 2 after reflector discussions.

[2020-08-21; Issue processing telecon: Option A is Tentatively Ready]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

Wording for both **Option A** and **Option B** are provided.

Option A:

1. Modify 23.3.4.7 [\[iterator.concept.sentinel\]](#) as indicated:

```
template<class S, class I>
concept sentinel_for =
    semiregular<S> &&
    input_or_output_iterator<I> &&
    weakly-equality-comparable-with<S, I>; // See 18.5.3 \[concept.equalitycomparable\]
```

-2- Let `s` and `i` be values of type `S` and `I` such that `[i, s)` denotes a range. Types `S` and `I` model `sentinel_for<S, I>` only if

- (2.1) — `i == s` is well-defined.
- (2.2) — If `bool(i != s)` then `i` is dereferenceable and `[++i, s)` denotes a range.

(2.?) — `assignable_from<I&, S>` is either modeled or not satisfied.

Option B:

1. Modify 23.4.4.2 [\[range.iter.op.advance\]](#) as indicated:

```
template<input_or_output_iterator I, sentinel_for<I> S>
constexpr void ranges::advance(I& i, S bound);
```

-3- *Preconditions*: `[i, bound)` denotes a range.

-4- *Effects*:

(4.1) — If `I` and `S` model ~~`assignable_from<I&, S>`~~ `same_as<I, S>`, equivalent to `i = std::move(bound)`.

(4.2) — [...]

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 23.3.4.7 [\[iterator.concept.sentinel\]](#) as indicated:

```
template<class S, class I>
concept sentinel_for =
    semiregular<S> &&
    input_or_output_iterator<I> &&
    weakly-equality-comparable-with<S, I>; // See 18.5.3 \[concept.equalitycomparable\]
```

-2- Let `s` and `i` be values of type `S` and `I` such that `[i, s)` denotes a range. Types `S` and `I` model `sentinel_for<S, I>` only if

(2.1) — `i == s` is well-defined.

(2.2) — If `bool(i != s)` then `i` is dereferenceable and `[++i, s)` denotes a range.

(2.2) — `assignable_from<I&, S>` is either modeled or not satisfied.

[3455](#). Incorrect *Postconditions* on `unique_ptr` move assignment

Section: 20.11.1.3.4 [\[unique.ptr.single.asgn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Howard Hinnant **Opened:** 2020-06-22 **Last modified:** 2020-09-06

Priority: 0

View all other [issues](#) in [\[unique.ptr.single.asgn\]](#).

Discussion:

20.11.1.3.4 [\[unique.ptr.single.asgn\]](#)/p5 says this about the `unique_ptr` move assignment operator:

Postconditions: `u.get() == nullptr`.

But this is only true if `this != &u`. For example:

```
#include <iostream>
#include <memory>

int main()
{
    auto x = std::unique_ptr<int>(new int{3});
    x = std::move(x);
    if (x)
        std::cout << *x << '\n';
    else
        std::cout << "nullptr\n";
}
```

Output:

3

An alternative resolution to that proposed below is to just delete the *Postcondition* altogether as the *Effects* element completely specifies everything. If we do that, then we should also remove `p10`, the *Postconditions* element for the converting move assignment operator. I have a slight preference for the proposed change below as it is more informative, at the expense of being a little more repetitive.

[2020-06-26; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after seven votes in favor during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 20.11.1.3.4 [\[unique.ptr.single.asgn\]](#) as indicated:

```
unique_ptr& operator=(unique_ptr&& u) noexcept;
```

-1- *Constraints*: `is_move_assignable_v<D>` is true.

-2- *Preconditions:* If `D` is not a reference type, `D` meets the *Cpp17MoveAssignable* requirements (Table [tab:cpp17.moveassignable]) and assignment of the deleter from an rvalue of type `D` does not throw an exception. Otherwise, `D` is a reference type; `remove_reference_t<D>` meets the *Cpp17CopyAssignable* requirements and assignment of the deleter from an lvalue of type `D` does not throw an exception.

-3- *Effects:* Calls `reset(u.release())` followed by `get_deleter() = std::forward<D>(u.get_deleter())`.

-4- *Returns:* `*this`.

-5- *Postconditions:* `If this != addressof(u), u.get() == nullptr, otherwise u.get() is unchanged.`

3460. Unimplementable `noop_coroutine_handle` guarantees

Section: 17.12.5.2.2 [coroutine.handle.noop.resumption] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2020-07-01 **Last modified:** 2020-09-06

Priority: 2

Discussion:

17.12.5.2.2 [coroutine.handle.noop.resumption]/2 states "Remarks: If `noop_coroutine_handle` is converted to `coroutine_handle<>`, calls to `operator()`, `resume` and `destroy` on that handle will also have no observable effects." This suggests that e.g. in this function:

```
void f(coroutine_handle<> meow)
{
    auto woof = noop_coroutine();
    static_cast<coroutine_handle<>&>(woof) = meow;
    static_cast<coroutine_handle<>&>(woof).resume();
}
```

the final call to `coroutine_handle<>::resume` must have no effect regardless of what coroutine (if any) `meow` refers to, contradicting the specification of `coroutine_handle<>::resume`. Even absent this contradiction, implementing the specification requires `coroutine_handle<>::resume` to determine if `*this` is a base subobject of a `noop_coroutine_handle`, which seems pointlessly expensive to implement.

17.12.5.2.4 [coroutine.handle.noop.address]/2 states "Remarks: A `noop_coroutine_handle`'s `ptr` is always a non-null pointer value." Similar to the above case, a slicing assignment of a default-initialized `coroutine_handle<>` to a `noop_coroutine_handle` must result in `ptr` having a null pointer value — another contradiction between the requirements of `noop_coroutine_handle` and `coroutine_handle<>`.

[2020-07-12; Reflector prioritization]

Set priority to 2 after reflector discussions.

[2020-07-29 Tim adds PR and comments]

The root cause for this issue as well as issue [3469](#) is the unnecessary public derivation from `coroutine_handle<void>`. The proposed resolution below replaces the derivation with a conversion function and adds explicit declarations for members that were previously inherited. It also modifies the preconditions on `from_address` with goal of making it impossible to obtain a `coroutine_handle<P>` to a coroutine whose promise type is not `P` in well-defined code.

[2020-08-21; Issue processing telecon: moved to Tentatively Ready]

Proposed resolution:

This wording is relative to [N4861](#) and also resolves LWG issue [3469](#).

1. Edit 17.12.4 [coroutine.handle] as indicated:

```
namespace std {
    [...]

    template<class Promise>
    struct coroutine_handle : coroutine_handle<>
    {
        // [coroutine.handle.con], construct/reset
        using coroutine_handle<>::coroutine_handle;
        constexpr coroutine_handle() noexcept;
        constexpr coroutine_handle(nullptr_t) noexcept;
        static coroutine_handle from_promise(Promise&);
        coroutine_handle& operator=(nullptr_t) noexcept;

        // [coroutine.handle.export.import], export/import
        constexpr void* address() const noexcept;
        static constexpr coroutine_handle from_address(void* addr);

        // [coroutine.handle.conv], conversion
        constexpr operator coroutine_handle<>() const noexcept;

        // [coroutine.handle.observers], observers
        constexpr explicit operator bool() const noexcept;
        bool done() const;

        // [coroutine.handle.resumption], resumption
        void operator()() const;
        void resume() const;
        void destroy() const;
    };
}
```

```

    // [coroutine.handle.promise], promise access
    Promise& promise() const;

private:
    void* ptr; // exposition only
};
}

```

-1- An object of type `coroutine_handle<T>` is called a *coroutine handle* and can be used to refer to a suspended or executing coroutine. A default-constructed `coroutine_handle` object whose member `address()` returns a null pointer value does not refer to any coroutine. Two `coroutine_handle` objects refer to the same coroutine if and only if their member `address()` returns the same value.

2. Add the following subclause under 17.12.4 [\[coroutine.handle\]](#), immediately after 17.12.4.2 [\[coroutine.handle.con\]](#):

??? Conversion [\[coroutine.handle.conv\]](#)

```
constexpr operator coroutine_handle<>() const noexcept;
```

-1- *Effects*: Equivalent to: `return coroutine_handle<>::from_address(address());`;

3. Edit 17.12.4.3 [\[coroutine.handle.export.import\]](#) as indicated, splitting the two versions:

```
static constexpr coroutine_handle<> coroutine_handle<>::from_address(void* addr);
```

-?- *Preconditions*: `addr` was obtained via a prior call to `address` on an object whose type is a specialization of `coroutine_handle`.

-?- *Postconditions*: `from_address(address()) == *this`.

```
static constexpr coroutine_handle<Promise> coroutine_handle<Promise>::from_address(void* addr);
```

-2- *Preconditions*: `addr` was obtained via a prior call to `address` on an object of type `cv coroutine_handle<Promise>`.

-3- *Postconditions*: `from_address(address()) == *this`.

4. Edit 17.12.5.2 [\[coroutine.handle.noop\]](#) as indicated:

```

namespace std {
    template<>
    struct coroutine_handle<noop_coroutine_promise> coroutine_handle<>
    {
        // [coroutine.handle.noop.conv], conversion
        constexpr operator coroutine_handle<>() const noexcept;

        // [coroutine.handle.noop.observers], observers
        constexpr explicit operator bool() const noexcept;
        constexpr bool done() const noexcept;

        // [coroutine.handle.noop.resumption], resumption
        constexpr void operator>() const noexcept;
        constexpr void resume() const noexcept;
        constexpr void destroy() const noexcept;

        // [coroutine.handle.noop.promise], promise access
        noop_coroutine_promise& promise() const noexcept;

        // [coroutine.handle.noop.address], address
        constexpr void* address() const noexcept;

    private:
        coroutine_handle(unspecified);
        void* ptr; // exposition only
    };
}

```

5. Add the following subclause under 17.12.5.2 [\[coroutine.handle.noop\]](#), immediately before 17.12.5.2.1 [\[coroutine.handle.noop.observers\]](#):

??? Conversion [\[coroutine.handle.noop.conv\]](#)

```
constexpr operator coroutine_handle<>() const noexcept;
```

-1- *Effects*: Equivalent to: `return coroutine_handle<>::from_address(address());`;

3461. convertible_to's description mishandles cv-qualified void

Section: 18.4.4 [\[concept.convertible\]](#) Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2020-07-03 Last modified: 2020-09-06

Priority: 0

View other [active issues](#) in [\[concept.convertible\]](#).

View all other [issues](#) in [\[concept.convertible\]](#).

Discussion:

There are no expressions of type cv-qualified void because any such expression must be prvalues and 7.2.2 [expr.type]/2 states:

If a prvalue initially has the type "cv T", where T is a cv-unqualified non-class, non-array type, the type of the expression is adjusted to T prior to any further analysis.

However, 18.4.4 [concept.convertible] p1 states:

Given types From and To and an expression E such that decltype((E)) is add_rvalue_reference_t<From>, convertible_to<From, To> requires E to be both implicitly and explicitly convertible to type To.

When From is cv-qualified void, E does not exist, yet we do want convertible_to<const void, void> to be modeled.

[2020-07-12; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after five votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 18.4.4 [concept.convertible] as indicated:

-1- Given types From and To and an expression E ~~such that decltype((E)) is add_rvalue_reference_t<From>~~ whose type and value category are the same as those of declval<From>(), convertible_to<From, To> requires E to be both implicitly and explicitly convertible to type To. The implicit and explicit conversions are required to produce equal results.

3465. compare_partial_order_fallback requires F < E

Section: 17.11.6 [cmp.alg] Status: [Tentatively Ready](#) Submitter: Stephan T. Lavavej Opened: 2020-07-18 Last modified: 2020-09-06

Priority: 0

View all other [issues](#) in [cmp.alg].

Discussion:

compare_partial_order_fallback uses three expressions, but requires only two. The decayed types of E and F are required to be identical, but variations in constness might make a difference.

[2020-07-26; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after seven votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 17.11.6 [cmp.alg] as indicated:

-6- The name compare_partial_order_fallback denotes a customization point object (16.3.3.3.6 [customization.point.object]). Given subexpressions E and F, the expression compare_partial_order_fallback(E, F) is expression-equivalent (3.21 [defs.expression-equivalent]) to:

(6.1) — If the decayed types of E and F differ, compare_partial_order_fallback(E, F) is ill-formed.

(6.2) — Otherwise, partial_order(E, F) if it is a well-formed expression.

(6.3) — Otherwise, if the expressions E == F, and E < F, and F < E are all both well-formed and convertible to bool,

```
E == F ? partial_ordering::equivalent :
E < F  ? partial_ordering::less      :
F < E  ? partial_ordering::greater   :
        partial_ordering::unordered
```

except that E and F are evaluated only once.

(6.4) — Otherwise, compare_partial_order_fallback(E, F) is ill-formed.

3466. Specify the requirements for promise/future/shared_future consistently

Section: 32.9.6 [futures.promise] Status: [Tentatively Ready](#) Submitter: Tomasz Kamiński Opened: 2020-07-18 Last modified: 2020-09-06

Priority: 3

View other [active issues](#) in [futures.promise].

View all other [issues](#) in [futures.promise].

Discussion:

The resolution of the LWG [3458](#) clearly specified the requirement that `future/shared_future` are ill-formed in situations when `T` is native array or function type. This requirement was not strictly necessary for `future<T>` as it was already ill-formed due the signature of the `get` function (that would be ill-formed in such case), however it was still added for consistency of specification. Similar, requirement should be introduced for the `promise<T>`, for which any call to `get_future()` would be ill-formed, if `T` is of array or function type.

[Note: `promise<int[10]>` is ill-formed for `libstdc++` and `libc++`, see [this code](#)]

[2020-07-26; Reflector prioritization]

Set priority to 3 after reflector discussions. Tim Song made the suggestion to replace the P/R wording by the following alternative wording:

For the primary template, `R` shall be an object type that meets the `Cpp17Destructible` requirements.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

Ideally the wording below would use a *Mandates*: element, but due to the still open issue LWG [3193](#) the wording below uses instead the more general "ill-formed" vocabulary.

1. Modify 32.9.6 [\[futures.promise\]](#) as indicated:

```
namespace std {
    template<class R>
    class promise {
        [...]
    };
    [...]
}
```

-?- If `is_array_v<R>` is true or `is_function_v<R>` is true, the program is ill-formed.

[2020-08-02; Daniel comments and provides alternative wording]

Following the suggestion of Tim Song a revised wording is provided which is intended to replace the currently agreed on wording for LWG [3458](#).

[2020-08-21; Issue processing telecon: Tentatively Ready]

Discussed a note clarifying that `Cpp17Destructible` disallows arrays (as well as types without accessible destructors). Can be added editorially.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 32.9.6 [\[futures.promise\]](#) as indicated:

```
namespace std {
    template<class R>
    class promise {
        [...]
    };
    [...]
}
```

-?- For the primary template, `R` shall be an object type that meets the `Cpp17Destructible` requirements.

-1- The implementation provides the template `promise` and two specializations, `promise<R&>` and `promise<void>`. These differ only in the argument type of the member functions `set_value` and `set_value_at_thread_exit`, as set out in their descriptions, below.

2. Modify 32.9.7 [\[futures.unique.future\]](#) as indicated:

```
namespace std {
    template<class R>
    class future {
        [...]
    };
}
```

-?- For the primary template, `R` shall be an object type that meets the `Cpp17Destructible` requirements.

-4- The implementation provides the template `future` and two specializations, `future<R&>` and `future<void>`. These differ only in the return type and return value of the member function `get`, as set out in its description, below.

3. Modify 32.9.8 [\[futures.shared.future\]](#) as indicated:

```
namespace std {
    template<class R>
    class shared_future {
        [...]
    };
}
```

-?- For the primary template, `R` shall be an object type that meets the *Cpp17Destructible* requirements.

-4- The implementation provides the template `shared_future` and two specializations, `shared_future<R&>` and `shared_future<void>`. These differ only in the return type and return value of the member function `get`, as set out in its description, below.

3467. `bool` can't be an integer-like type

Section: 23.3.4.4 [\[iterator.concept.winc\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2020-07-23 **Last modified:** 2020-09-06

Priority: 0

View other [active issues](#) in [\[iterator.concept.winc\]](#).

View all other [issues](#) in [\[iterator.concept.winc\]](#).

Discussion:

Per 24.2 [\[ranges.syn\]/1](#), the Standard Library believes it can convert an integer-like type `x` to an unsigned integer-like type with the exposition-only type alias *make-unsigned-like-t*. *make-unsigned-like-t<x>* is specified as being equivalent to `make_unsigned_t<x>` when `x` is an integral type. However, despite being an integral type, `bool` is not a valid template type argument for `make_unsigned_t` per [\[tab:meta.trans.sign\]](#).

This problem with `bool` was an oversight when we added support for integer-like types: it was certainly not the design intent to allow `ranges::size(r)` to return `false`! While we could devise some more-complicated metaprogramming to allow use of `bool`, it seems easier — and consistent with the design intent — to simply exclude `bool` from the set of integer-like types.

[2020-08-02; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after six votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 23.3.4.4 [\[iterator.concept.winc\]](#) as indicated:

-11- A type `T` **other than `cv bool`** is *integer-like* if it models `integral<I>` or if it is an integer-class type. An *integer-like* type `T` is *signed-integer-like* if it models `signed_integral<I>` or if it is a signed-integer-class type. An *integer-like* type `T` is *unsigned-integer-like* if it models `unsigned_integral<I>` or if it is an unsigned-integer-class type.

3472. `counted_iterator` is missing preconditions

Section: 23.5.6 [\[iterators.counted\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Michael Schellenberger Costa **Opened:** 2020-07-29 **Last modified:** 2020-09-06

Priority: Not Prioritized

Discussion:

C++20 introduces a new iterator `counted_iterator` that keeps track of the end of its range via an additional exposition only member `length`.

Consequently, there are several preconditions for many member functions of `counted_iterator`, but it seems some are missing:

1. `operator*`

Here we have no precondition regarding `length`. However, given that `length` denotes the distance to the end of the range it should be invalid to dereference a `counted_iterator` with `length 0`.

Moreover, `operator[]` has a precondition of `"n < length"`. Consider the following code snippet:

```
int some_ints[] = {0,1,2};
counted_iterator<int*> i{some_ints, 0};
```

Here `"i[0]"` would be invalid due to the precondition `"n < length"`. However, `"*i"` would be a valid expression. This violates the definition of `operator[]` which states according to 7.6.1.2 [\[expr.sub\]](#) p1:

[...] The expression `E1[E2]` is identical (by definition) to `*((E1)+(E2))` [...]

Substituting `E2->0` we get

[...] The expression `E1[0]` is identical (by definition) to `*(E1)` [...]

With the current wording `counted_iterator` violates that definition and we should add to `operator*`:

Preconditions: `length > 0`.

2. `iter_move`

This is a similar case. We have only the *Effects* element:

Effects: Equivalent to: `return ranges::iter_move(i.current);`

However, looking at the requirements of `ranges::iter_move` we have in 23.3.3.1 [\[iterator.cust.move\]](#) p2:

If `ranges::iter_move(E)` is not equal to `*E`, the program is ill-formed, no diagnostic required.

This clearly requires that for `counted_iterator::iter_move` to be well-formed, we need `counted_iterator::operator*` to be well formed. Consequently we should also add the same precondition to `counted_iterator::iter_move`:

Preconditions: `length > 0`.

3. `iter_swap`

This is essentially the same arguing as for `counted_iterator::iter_move`. The essential observation is that `ranges::iter_swap` is defined in terms of `ranges::iter_move` (see 23.3.3.2 [\[iterator.cust.swap\]](#)) so it must have the same preconditions and we should add:

Preconditions: `length > 0`.

[2020-08-21 Issue processing telecon: moved to Tentatively Ready.]

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 23.5.6.4 [\[counted.iter.elem\]](#) as indicated:

```
constexpr decltype(auto) operator*();
constexpr decltype(auto) operator*() const
    requires dereferenceable<const I>;
```

-?- Preconditions: `length > 0`.

-1- *Effects:* Equivalent to: `return *current;`

2. Modify 23.5.6.7 [\[counted.iter.cust\]](#) as indicated:

```
friend constexpr iter_rvalue_reference_t<I>
    iter_move(const counted_iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current)))
    requires input_iterator<I>;
```

-?- Preconditions: `i.length > 0`.

-1- *Effects:* Equivalent to: `return ranges::iter_move(i.current);`

```
template<indirectly_swappable<I> I2>
    friend constexpr void
        iter_swap(const counted_iterator& x, const counted_iterator<I2>& y)
        noexcept(noexcept(ranges::iter_swap(x.current, y.current)));
```

-?- Preconditions: `x.length > 0` and `y.length > 0`.

-1- *Effects:* Equivalent to: `return ranges::iter_swap(x.current, y.current);`

[3473](#). Normative encouragement in non-normative note

Section: 20.20.6.3 [\[format.args\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2020-07-31 **Last modified:** 2020-09-06

Priority: 0

Discussion:

The note in the final paragraph of 20.20.6.3 [\[format.args\]](#) gives encouragement to implementations, which is not allowed in a note.

It needs to be normative text, possibly using "should", or if left as a note could be phrased as "Implementations can optimize the representation [...]".

[2020-08-09; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after six votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 20.20.6.3 [\[format.args\]](#) as indicated:

-1- An instance of `basic_format_args` provides access to formatting arguments. **Implementations should optimize the representation of `basic_format_args` for a small number of formatting arguments. [Note: For example, by storing indices of type alternatives separately from values and packing the former. — end note]**

`basic_format_args()` `noexcept`;

-2- *Effects*: Initializes `size_` with 0.

[...]

`basic_format_arg<Context> get(size_t i) const noexcept;`

-4- *Returns*: `i < size_ ? data_[i] : basic_format_arg<Context>()`.

[Note: Implementations are encouraged to optimize the representation of `basic_format_args` for small number of formatting arguments by storing indices of type alternatives separately from values and packing the former. — *end note*]

3474. Nesting `join_views` is broken because of CTAD

Section: 24.7.11 [\[range.join\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Barry Revzin **Opened:** 2020-08-04 **Last modified:** 2020-09-06

Priority: Not Prioritized

Discussion:

Let's say I had a range of range of ranges and I wanted to recursively flatten it. That would involve repeated invocations of `join`. But this doesn't work:

```
std::vector<std::vector<std::vector<int>>> nested_vectors = {
    {{1, 2, 3}, {4, 5}, {6}},
    {{7},      {8, 9}, {10, 11, 12}},
    {{13}}
```

```
};
auto joined = nested_vectors | std::views::join | std::views::join;
```

The expectation here is that the `value_type` of `joined` is `int`, but it's actually `vector<int>` — because the 2nd invocation of `join` ends up just copying the first. This is because `join` is specified to do:

The name `views::join` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given a subexpression `E`, the expression `views::join(E)` is expression-equivalent to `join_view{E}`.

And `join_view{E}` for an `E` that's already a specialization of a `join_view` just gives you the same `join_view` back. Yay CTAD. We need to do the same thing with `join` that we did with `reverse` in [P1252](#). We can do that either in exposition (**Option A**) by modifying 24.7.11.1 [\[range.join.overview\]](#) p2

The name `views::join` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given a subexpression `E`, the expression `views::join(E)` is expression-equivalent to `join_view<views::all_t<decltype((E))>>{E}`.

Or in code (**Option B**) add a deduction guide to 24.7.11.2 [\[range.join.view\]](#):

```
template<class R>
explicit join_view(R&&) -> join_view<views::all_t<R>>;

template<class V>
explicit join_view(join_view<V>) -> join_view<join_view<V>>;
```

[2020-08-21; Issue processing telecon: Option A is Tentatively Ready]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

[Drafting Note: Two mutually exclusive options are prepared, depicted below by **Option A** and **Option B**, respectively.]

Option A:

1. Modify 24.7.11.1 [\[range.join.overview\]](#) as indicated:

-2- The name `views::join` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given a subexpression `E`, the expression `views::join(E)` is expression-equivalent to `join_view<views::all_t<decltype((E))>>{E}`.

Option B:

1. Modify 24.7.11.2 [\[range.join.view\]](#) as indicated:

```
namespace std::ranges {
    [...]

    template<class R>
    explicit join_view(R&&) -> join_view<views::all_t<R>>;

    template<class V>
    explicit join_view(join_view<V>) -> join_view<join_view<V>>;
}
```

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 24.7.11.1 [\[range.join.overview\]](#) as indicated:

-2- The name `views::join` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given a subexpression `E`, the expression `views::join(E)` is expression-equivalent to `join_view<views::all_t<decltype((E))>>{E}`.

3476. thread and jthread constructors require that the parameters be move-constructible but never move construct the parameters

Section: 32.4.3.3 [\[thread.thread.constr\]](#), 32.4.4.2 [\[thread.jthread.cons\]](#), 32.9.9 [\[futures.async\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Billy O'Neal III **Opened:** 2020-08-18 **Last modified:** 2020-09-06

Priority: 0

View other [active issues](#) in [\[thread.thread.constr\]](#).

View all other [issues](#) in [\[thread.thread.constr\]](#).

Discussion:

I think this was upgraded to *Mandates* because C++17 and earlier had "F and each T_i in Args shall satisfy the *Cpp17MoveConstructible* requirements." And for those, I think the requirement was attempting to make the subsequent *decay-copy* valid. However, the 'Mandating the standard library' papers added *is_constructible* requirements which already serve that purpose; `std::(j)thread` has no reason to move the elements after they have been *decay-copy*'d to transfer to the launched thread.

[2020-08-26; Reflector discussion]

Jonathan noticed that the wording for `std::async` is affected by exactly the same unnecessary move-constructible requirements. The proposed wording has been updated to cope for that as well.

[2020-09-02; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after five votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 32.4.3.3 [\[thread.thread.constr\]](#) as indicated:

```
template<class F, class... Args> explicit thread(F&& f, Args&&... args);
```

-3- *Constraints:* `remove_cvref_t<F>` is not the same type as `thread`.

-4- *Mandates:* The following are all true:

(4.1) — `is_constructible_v<decay_t<F>, F>`,

(4.2) — `(is_constructible_v<decay_t<Args>, Args> && ...)`, **and**

~~(4.3) — `is_move_constructible_v<decay_t<F>>`;~~

~~(4.4) — `(is_move_constructible_v<decay_t<Args>> && ...)`, and~~

(4.5) — `is_invocable_v<decay_t<F>, decay_t<Args>...>`.

~~5- *Preconditions:* `decay_t<F>` and each type in `decay_t<Args>` meet the *Cpp17MoveConstructible* requirements.~~

2. Modify 32.4.4.2 [\[thread.jthread.cons\]](#) as indicated:

```
template<class F, class... Args> explicit jthread(F&& f, Args&&... args);
```

-3- *Constraints:* `remove_cvref_t<F>` is not the same type as `jthread`.

-4- *Mandates:* The following are all true:

(4.1) — `is_constructible_v<decay_t<F>, F>`,

(4.2) — `(is_constructible_v<decay_t<Args>, Args> && ...)`, **and**

~~(4.3) — `is_move_constructible_v<decay_t<F>>`;~~

~~(4.4) — `(is_move_constructible_v<decay_t<Args>> && ...)`, and~~

(4.5) — `is_invocable_v<decay_t<F>, decay_t<Args>...> || is_invocable_v<decay_t<F>, stop_token, decay_t<Args>...>`.

~~5- *Preconditions:* `decay_t<F>` and each type in `decay_t<Args>` meet the *Cpp17MoveConstructible* requirements.~~

3. Modify 32.9.9 [\[futures.async\]](#) as indicated:

```
template<class F, class... Args>
[[nodiscard]] future<invoke_result_t<decay_t<F>, decay_t<Args>...>
  async(F&& f, Args&&... args);
template<class F, class... Args>
[[nodiscard]] future<invoke_result_t<decay_t<F>, decay_t<Args>...>
  async(launch policy, F&& f, Args&&... args);
```

-2- Mandates: The following are all true:

(2.1) — `is_constructible_v<decay_t<F>, F>`,

(2.2) — `(is_constructible_v<decay_t<Args>, Args> && ...)`, **and**

~~(2.3) — `is_move_constructible_v<decay_t<F>>`,~~

~~(2.4) — `(is_move_constructible_v<decay_t<Args>> && ...)`, and~~

(2.5) — `is_invocable_v<decay_t<F>, decay_t<Args>...>`.

~~3- Preconditions: `decay_t<F>` and each type in `decay_t<Args>` meet the Cpp17 MoveConstructible requirements.~~

3477. Simplify constraints for *semiregular-box*

Section: 24.7.3 [\[range.semi.wrap\]](#) Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2020-08-19 Last modified: 2020-09-06

Priority: 0

View other [active issues](#) in [\[range.semi.wrap\]](#).

View all other [issues](#) in [\[range.semi.wrap\]](#).

Discussion:

The exposition-only *semiregular-box* class template specified in 24.7.3 [\[range.semi.wrap\]](#) implements a default constructor, copy assignment operator, and move assignment operator atop the facilities provided by `std::optional` when the wrapped type is not default constructible, copy assignable, or move assignable (respectively). The constraints on the copy and move assignment operator implementations go out of their way to be unnecessarily minimal. The meaning of the constraint on the copy assignment operator — `!assignable<T, const T>` — has even changed since this wording was written as a result of LWG reformulating the implicit expression variations wording in 18.2 [\[concepts.equality\]](#).

It would be much simpler for implementors and users if we recall that minimality is not the primary goal of constraints and instead constrain these assignment operators more simply with `!movable<T>` and `!copyable<T>`.

[2020-09-03; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after six votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 24.7.3 [\[range.semi.wrap\]](#) as indicated:

-1- Many types in this subclause are specified in terms of an exposition-only class template *semiregular-box*. *semiregular-box*<T> behaves exactly like `optional<T>` with the following differences:

(1.1) — [...]

(1.2) — [...]

(1.3) — If ~~`assignable_from<T&, const T>`~~ `copyable<T>` is not modeled, the copy assignment operator is equivalent to:

```
semiregular-box& operator=(const semiregular-box& that)
  noexcept(is_nothrow_copy_constructible_v<T>)
{
  if (that) emplace(*that);
  else reset();
  return *this;
}
```

(1.4) — If ~~`assignable_from<T&, T>`~~ `movable<T>` is not modeled, the move assignment operator is equivalent to:

```
semiregular-box& operator=(semiregular-box&& that)
  noexcept(is_nothrow_move_constructible_v<T>)
{
  if (that) emplace(std::move(*that));
  else reset();
  return *this;
}
```

3482. drop_view's const begin should additionally require sized_range

Section: 24.7.9.2 [\[range.drop.view\]](#) Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2020-08-31 Last modified: 2020-09-29

Priority: 0

Discussion:

When the underlying range models both `random_access_range` and `sized_range`, a `drop_view` can easily calculate its first iterator in $\mathcal{O}(1)$ as by the underlying range's first iterator plus the minimum of the number of elements to drop and the size of the underlying range. In this case `drop_view::begin` need not "cache the result within the `drop_view` for use on subsequent calls" "in order to provide the amortized constant-time complexity required by the range concept" (24.7.9.2 [\[range.drop.view\]](#)/4). However, `drop_view::begin()` `const` does not require `sized_range`, it requires only `random_access_range`. There's no way to implementing what amounts to a requirement that calls to `begin` after the first must be $\mathcal{O}(1)$ without memoization.

Performing memoization in a `const` member function in a manner consistent with 16.4.6.10 [\[res.on.data.races\]](#) is impossible without some kind of thread synchronization. It is not the intended design for anything in current Range library to require such implementation heroics, we typically fall back to mutable-only iteration to avoid thread synchronization concerns. (Note that both `range-v3` and `cmcstl2` handle `drop_view::begin()` `const` incorrectly by performing $\mathcal{O}(N)$ lookup of the first iterator on each call to `begin`, which is consistent with 16.4.6.10 [\[res.on.data.races\]](#) but fails to meet the complexity requirements imposed by the range concept.) We should fall back to mutable-only iteration here as well when the underlying range is not a `sized_range`.

For `drop_view`, changing the constraints on the `const` overload of `begin` also requires changing the constraints on the non-`const` overload. The non-`const` `begin` tries to constrain itself out of overload resolution when the `const` overload would be valid if the underlying range models the exposition-only `simple-view` concept. (Recall that `T` models `simple-view` iff `T` models `view`, `const T` models `range`, and `T` and `const T` have the same iterator and sentinel types.) Effectively this means the constraints on the non-`const` overload must require either that the underlying range fails to model `simple-view` or that the constraints on the `const` overload would not be satisfied. So when we add a new `sized_range` requirement to the `const` overload, we must also add its negation to the mutable overload. (The current form of the constraint on the mutable `begin` overload is `!(simple-view<V> && random_access_range<V>)` instead of `!(simple-view<V> && random_access_range<const V>)` because of an unstated premise that `V` and `const V` should both have the same category when both are ranges. Avoiding this unstated premise would make it easier for future readers to grasp what's happening here; we should formulate our new constraints in terms of `const V` instead of `V`.)

[2020-09-29; Reflector discussions]

Status to Tentatively Ready and priority to 0 after five positive votes on the reflector.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 24.7.9.2 [\[range.drop.view\]](#) as indicated:

```
namespace std::ranges {
    template<view V>
    class drop_view : public view_interface<drop_view<V>> {
    public:
        [...]
        constexpr auto begin()
            requires (!(simple-view<V> &&
                random_access_range<const V> && sized_range<const V>));
        constexpr auto begin() const
            requires random_access_range<const V> && sized_range<const V>;
        [...]
    };
}
```

[...]

```
constexpr auto begin()
    requires !(simple-view<V> &&
        random_access_range<const V> && sized_range<const V>);
constexpr auto begin() const
    requires random_access_range<const V> && sized_range<const V>;
```

-3- *Returns:* `ranges::next(ranges::begin(base_), count_, ranges::end(base_))`.

-4- *Remarks:* In order to provide the amortized constant-time complexity required by the range concept when `drop_view` models `forward_range`, the first overload caches the result within the `drop_view` for use on subsequent calls. [Note: Without this, applying a `reverse_view` over a `drop_view` would have quadratic iteration complexity. — end note]

3483. transform_view::iterator's difference is overconstrained

Section: 24.7.6.3 [\[range.transform.iterator\]](#), 24.7.16.3 [\[range.elements.iterator\]](#) Status: [Tentatively Ready](#) Submitter: Casey Carter Opened: 2020-09-04 Last modified: 2020-09-13

Priority: 0

View all other [issues](#) in [\[range.transform.iterator\]](#).

Discussion:

The difference operation for `transform_view::iterator` is specified in 24.7.6.3 [\[range.transform.iterator\]](#) as:


```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
requires random_access_range<Base>;
```

-22- *Effects*: Equivalent to: return `x.current_ - y.current_;`

The member `current_` is an iterator of type `iterator_t<Base>`, where `Base` is `V` for `transform_view<V, F>::iterator<false>` and `const V` for `transform_view<V, F>::iterator<true>`. The difference of iterators that appears in the above *Effects*: element is notably well-defined if their type models `sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>` which `random_access_range<Base>` refines. This overstrong requirement seems to be simply the result of an oversight; it has been present since [P0789R0](#), without — to my recollection — ever having been discussed. We should relax this requirement to provide difference capability for `transform_view`'s iterators whenever the underlying iterators do.

[2020-09-08; Reflector discussion]

During reflector discussions it was observed that `elements_view::iterator` has the same issue and the proposed wording has been extended to cover this template as well.

[2020-09-13; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after seven votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 24.7.6.3 [\[range.transform.iterator\]](#) as indicated:

```
namespace std::ranges {
    template<input_range V, copy_constructible F>
        requires view<V> && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<V>> &&
                can_reference_invoke_result_t<F&, range_reference_t<V>>>
        template<bool Const>
        class transform_view<V, F>::iterator {
        public:
            [...]
            friend constexpr iterator operator-(iterator i, difference_type n)
                requires random_access_range<Base>;
            friend constexpr difference_type operator-(const iterator& x, const iterator& y)
                requires random_access_range<Base> sized_sentinel_for<iterator t<Base>, iterator t<Base>>;
            [...]
        };
}

[...]

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
requires random_access_range<Base> sized_sentinel_for<iterator t<Base>, iterator t<Base>>;

-22- Effects: return x.current_ - y.current_;
```

2. Modify 24.7.16.3 [\[range.elements.iterator\]](#) as indicated:

```
namespace std::ranges {
    template<input_range V, size_t N>
        requires view<V> && has_tuple_element<range_value_t<V>, N> &&
            has_tuple_element<remove_reference_t<range_reference_t<V>>, N>
        template<bool Const>
        class elements_view<V, N>::iterator { // exposition only
        [...]
            friend constexpr iterator operator-(iterator x, difference_type y)
                requires random_access_range<Base>;
            friend constexpr difference_type operator-(const iterator& x, const iterator& y)
                requires random_access_range<Base> sized_sentinel_for<iterator t<Base>, iterator t<Base>>;
        };
}

[...]

constexpr difference_type operator-(const iterator& x, const iterator& y)
requires random_access_range<Base> sized_sentinel_for<iterator t<Base>, iterator t<Base>>;

-21- Effects: return x.current_ - y.current_;
```

Tentatively NAD Issues

These issues were moved to **Tentatively NAD** since the Prague meeting, either during LWG teleconferences or following reflector polls.

[1396](#). regex should support allocators

Section: 30.7 [\[re.regex\]](#) Status: [Tentatively NAD](#) Submitter: INCITS Opened: 2010-08-25 Last modified: 2020-09-06

Priority: Not Prioritized

View other [active issues](#) in [re.regex].

View all other [issues](#) in [re.regex].

Duplicate of: [1451](#)

Discussion:

Addresses US-104, US-141

std::basic_regex should have an allocator for all the reasons that a std::string does. For example, I can use boost::interprocess to put a string or vector in shared memory, but not a regex.

[Resolution proposed by ballot comment]

Add allocators to regexes

[2010-10-24 Daniel adds:]

Accepting [n3171](#) would solve this issue.

[2011-03-22 Madrid]

Close 1396 as NAD Future.

[LEWG Kona 2017]

Recommend Open: Covered in LEWG9, P0269, which is in wording review.

[2020-07-17; status changed to Tentatively NAD in issue processing telecon]

P0269R0 has been superseded by P1294R0. Requested LEWG to confirm which version they forwarded to LWG. In any case, this is a feature request, not a defect, and will be dealt with as a proposal not an issue.

Rationale:

No consensus for a change at this time

Proposed resolution:

[2335](#). array<array<int, 3>, 4> should be layout-compatible with int[4][3]

Section: 22.3.7 [\[array\]](#) Status: [Tentatively NAD](#) Submitter: Jeffrey Yasskin Opened: 2013-10-04 Last modified: 2020-02-16

Priority: 3

View all other [issues](#) in [array].

Discussion:

In order to replace some uses of C arrays with std::array, we need it to be possible to cast from a std::array<> to an equivalent C array. Core wording doesn't appear to be in quite the right state to allow casting, but if we specify that appropriate types are layout-compatible, we can at least write:

```
union {
    array<array<array<int, 2>, 3>, 4> arr;
    int carr[4][3][2];
};
```

to view memory as the other type: C++14 CD [class.mem]p18.

I believe it's sufficient to add "array<T, N> shall be layout-compatible (6.8 [\[basic.types\]](#)) with T[N]." to 22.3.7.1 [\[array.overview\]](#), but we might also need some extension to 11.4 [\[class.mem\]](#) to address the possibility of layout-compatibility between struct and array types.

I checked that libc++ on MacOS already implements this, although it would be good for someone else to double-check; I haven't checked any other standard libraries.

[2020-02-14, Prague]

LWG discussions and decision for NAD.

Rationale:

The desire to use std::array like this seems like an "XY problem". The goal should be "replace C arrays" not "replace C arrays with std::array", because std::array is not suitable here. There are superior solutions being proposed, and will be available in a future version of C++ (e.g. using `std::span` as a multi-dimensional view on an array).

Proposed resolution:

[3207](#). N in `ssize(const T (&)[N])` should be `size_t`

Section: 23.7 [\[iterator.range\]](#) **Status:** [Tentatively NAD](#) **Submitter:** Nevin Liber **Opened:** 2019-05-23 **Last modified:** 2020-09-06

Priority: Not Prioritized

View other [active issues](#) in [\[iterator.range\]](#).

View all other [issues](#) in [\[iterator.range\]](#).

Discussion:

The `N` in `ssize(const T (&)[N])` is specified to be of type `ptrdiff_t`. It should be `size_t` to be consistent with the rest of the standard library, such as the array overloads for all other range access functions, the swap overload for arrays, and other function template overloads for arrays. (Note: the return type of this function should still be `ptrdiff_t`.)

[2019-06-12 Tentatively NAD after reflector discussion]

Proposed resolution:

This wording is relative to [N4810](#).

1. Modify 23.7 [\[iterator.range\]](#) as indicated:

```
template<class T, ptrdiff_size_t N> constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
```

-19- Returns: `static_cast<ptrdiff_t>(N)`.

[3365](#). Rename *ref-is-glvalue* to *deref-is-ref*

Section: 24.7.11.3 [\[range.join.iterator\]](#) **Status:** [Tentatively NAD](#) **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2020-01-07 **Last modified:** 2020-01-14

Priority: 0

View all other [issues](#) in [\[range.join.iterator\]](#).

Discussion:

The name of `join_view::iterator::ref-is-glvalue`, defined as `is_reference_v<range_reference_t<Base>>`, doesn't take into account the fact that it may also be true because `range_reference_t<Base>` is an rvalue (e.g. for `move_iterator<int>`).

We suggest renaming it to *deref-is-ref*.

[2020-01-14 Status set to Tentatively NAD after five positive votes on the reflector. This issue was based on a misunderstanding by the submitter which had been cleared up in an [related editorial issue](#).]

Proposed resolution:

Rename all occurrences referring to the symbol `join_view::iterator::ref-is-glvalue` to *deref-is-ref*.

[3394](#). `ranges::basic_istream_view::iterator` has an empty `iterator_traits`

Section: 24.6.5.3 [\[range.istream.iterator\]](#) **Status:** [Tentatively NAD](#) **Submitter:** Patrick Palka **Opened:** 2020-02-09 **Last modified:** 2020-02-14

Priority: 2

View other [active issues](#) in [\[range.istream.iterator\]](#).

View all other [issues](#) in [\[range.istream.iterator\]](#).

Discussion:

Every instantiation of `ranges::basic_istream_view::iterator` has an empty `iterator_traits`, i.e. the type

```
iterator_traits<ranges::basic_istream_view<Val, CharT, Traits>::iterator>
```

has no members.

This happens because `basic_istream_view::iterator` neither models *cpp17-iterator* (since this concept requires copyability, which this iterator is by design not) nor does it define all four of the member types `difference_type`, `value_type`, `reference`, and `iterator_category` (it is missing `reference`). Therefore by 23.3.2.3 [\[iterator.traits\]](#) p3, this iterator's specialization of `iterator_traits` will be empty if generated from the `iterator_traits` primary template.

Since `basic_istream_view::iterator` is indeed an iterator, its `iterator_traits` should certainly not be empty. The simplest solution here is to define the member type `reference` in the definition of `basic_istream_view::iterator`, which will enable its `iterator_traits` specialization to be appropriately populated from the primary template.

[2020-02-10; Jonathan comments]

Jonathan and Casey have concerns about the proposed resolution. Casey: The wording makes it look as if this iterator is supposed to be an *cpp17-input-iterator*.

See also LWG [3283](#) and [3289](#).

[2020-02 Prioritized as P2 Monday morning in Prague]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4849](#).

1. Modify 24.6.5.3 [\[range.istream.iterator\]](#), class template `basic_istream_view::iterator` synopsis, as indicated:

```
namespace std::ranges {
    template<class Val, class CharT, class Traits>
    class basic_istream_view<Val, CharT, Traits>::iterator { // exposition only
    public:
        using iterator_category = input_iterator_tag;
        using difference_type = ptrdiff_t;
        using value_type = Val;
        using reference = Val&;

        iterator() = default;
        [...]
    };
}
```

[2020-02-13, Prague]

LWG decided for NAD: The `ranges::basic_istream_view::iterator` is a move-only type and thus cannot meet the *Cpp17* requirements (even `output_iterator_tag`), as such it should not specialize `iterator_traits`, to avoid misleading results when it is passed to new algorithms.

A related issue, LWG [3397](#), is supposed to take care for a problem with the definition of the `iterator_category` member type of this template.

Proposed resolution:

[3399](#). `basic_syncbuf::emit()` + Qt's `#define emit` = Big Bada-Boom

Section: 29.10.2.1 [\[syncstream.syncbuf.overview\]](#) **Status:** [Tentatively NAD](#) **Submitter:** Marc Mutz **Opened:** 2019-02-14 **Last modified:** 2020-09-06

Priority: Not Prioritized

View all other [issues](#) in [\[syncstream.syncbuf.overview\]](#).

Discussion:

The current IS contains a function called `emit()` (in `basic_syncbuf`, added by [P0053](#)).

`emit` is a macro in pervasive use in every [Qt program](#). `#include'ing <osyncstream>` after any Qt header would break, because `emit` was `#define'd` to nothing by the Qt headers.

It is understood that the committee cannot check every 3rd-party library out there that chooses (badly, as I'd readily concur) to `#define` a macro of all-lowercase letters, but the min/max issue in the Windows headers caused so much pain for our users, that we probably should avoid a breakage here, for the benefit of our users that have to work Qt.

It also doesn't seem like `emit()` is a particularly mandatory name for the `syncbuf` function. Since it returns `bool`, it could just as easily be called `try_emit()` and the issue would be solved.

Suggested approach:

In `basic_syncbuf`, rename `bool emit()` to `bool try_emit()`. In `basic_osyncstream`, where the function doesn't return `bool`, but sets the stream's failbit, rename `void emit()` to `void emit_or_fail()`.

[2020-02-14, Prague]

The issue was sent to LEWG, who made the following poll:

We're open to renaming `osyncstream::emit()` (and related).

```
SF F N A SA
1 1 5 8 20
```

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 29.7.5.5 [\[ostream.manip\]](#) as indicated:

```
template<class charT, class traits>
    basic_ostream<charT, traits>& flush_emit(basic_ostream<charT, traits>& os);

-12- Effects: Calls os.flush(). Then, if os.rdbuf() is a basic_syncbuf<charT, traits, Allocator>*, called buf for the
purpose of exposition, calls buf->try_emit().
```

2. Modify 29.10.2.1 [\[syncstream.syncbuf.overview\]](#) as indicated:

```
[...]
// 29.10.2.4 \[syncstream.syncbuf.members\], member functions
bool try_emit();
streambuf_type* get_wrapped() const noexcept;
[...]
```

-1- Class template `basic_syncbuf` stores character data written to it, known as the associated output, into internal buffers allocated using the object's allocator. The associated output is transferred to the wrapped stream buffer object `*wrapped` when `try_emit()` is called or when the `basic_syncbuf` object is destroyed. Such transfers are atomic with respect to transfers by other `basic_syncbuf` objects with the same wrapped stream buffer object.

3. Modify 29.10.2.2 [\[syncstream.syncbuf.cons\]](#) as indicated:

~basic_syncbuf();

-7- Effects: Calls `try_emit()`.

-8- Throws: Nothing. If an exception is thrown from `try_emit()`, the destructor catches and ignores that exception.

4. Modify 29.10.2.3 [\[syncstream.syncbuf.assign\]](#) as indicated:

basic_syncbuf& operator=(basic_syncbuf&& rhs) noexcept;

-1- Effects: Calls `try_emit()` then move assigns from `rhs`. After the move assignment `*this` has the observable state it would have had if it had been move constructed from `rhs` (29.10.2.2 [\[syncstream.syncbuf.cons\]](#)).

5. Replace in 29.10.2.4 [\[syncstream.syncbuf.members\]](#) all occurrences of `emit()` by `try_emit()` (five occurrences)

6. Replace in 29.10.2.5 [\[syncstream.syncbuf.virtuals\]](#) all occurrences of `emit()` by `try_emit()` (three occurrences)

7. Modify 29.10.3.1 [\[syncstream.osyncstream.overview\]](#) as indicated:

```
[...]
// 29.10.3.3 \[syncstream.osyncstream.members\], member functions
void emit_or_fail();
streambuf_type* get_wrapped() const noexcept;
[...]
```

8. Replace in 29.10.3.2 [\[syncstream.osyncstream.cons\]](#) all occurrences of `emit()` by `emit_or_fail()` (three occurrences)

9. Replace in [\[syncstream.osyncstream.assign\]](#) all occurrences of `emit()` by `emit_or_fail()` (two occurrences)

10. Replace in 29.10.3.3 [\[syncstream.osyncstream.members\]](#) all occurrences of `emit()` by `emit_or_fail()` (six occurrences)

3440. Aggregate-paren-init breaks direct-initializing a tuple or optional from {aggregate-member-value}

Section: 20.5.3.1 [\[tuple.cnstr\]](#), 20.6.3.2 [\[optional.ctor\]](#) Status: [Tentatively NAD](#) Submitter: Ville Voutilainen Opened: 2020-05-01 Last modified: 2020-06-27

Priority: 2

View other [active issues](#) in [\[tuple.cnstr\]](#).

View all other [issues](#) in [\[tuple.cnstr\]](#).

Discussion:

For reference, see [this gcc bug report](#).

Constructing a tuple or optional from an element value of an aggregate is broken in C++20. `tuple<c> t({val});` and `optional<c> t({val});` invoked a non-forwarding constructor before, but now the perfect-forwarding converting constructors are a match, because the element is constructible from `{val}`. But it's not convertible, so overload resolution chooses the explicit constructor, and the initialization fails.

Tim Song explains the overload resolution in [this reflector discussion](#).

Now that we understand that C++17 called the non-forwarding conversion constructor, and C++20 tries to use the forwarding conversion constructor, we have the solution. SFINAE away the forwarding conversion constructor when it would convert an aggregate.

This also means that `tuple<c> t{};` won't work, which is unfortunate because `tuple<c>/optional<c>` no longer mirrors what `c` can do. That's okay; in this LWG issue, we first restore feature parity with C++17, and later, as an extension, enable such initializations so that `tuple/optional` mirrors what `c` can do in C++20.

The proposed wording below has been implemented and tested.

[2020-05-09; Reflector prioritization]

Set priority to 2 after reflector discussions.

[2020-06-11; LWG Telecon: Status changed: New → LEWG.]

Ask LEWG if it's desirable to make (`{val}`) work again. Tomasz would prefer it to be explicit e.g. via `std::in_place`.

[2020-06-23; LEWG Telecon]

POLL: Make (`{val}`) work again, at the risk of non-transparency of tuple constructors and further complicating the tuple and optional overload set.

SF F N A SA
0 5 6 9 0

No consensus for change. Close as Not a defect.

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 20.5.3.1 [\[tuple.cnstr\]](#) as indicated:

```
template<class... UTypes> constexpr explicit(see below) tuple(UTypes&&... u);
```

-11- *Constraints:* `sizeof...(Types)` equals `sizeof...(UTypes)` and `sizeof...(Types) ≥ 1` and `is_constructible_v<Ti, Ui>` is true for all `i` and `conjunction_v<is_aggregate<remove_reference_t<Ti>>, negation<is_same<remove_reference_t<Ti>, remove_reference_t<Ui>>>>` is false for all `i`.

2. Modify 20.6.3.2 [\[optional.ctor\]](#) as indicated:

```
template<class U = T> constexpr explicit(see below) optional(U&& v);
```

-22- *Constraints:* `is_constructible_v<T, U>` is true, `is_same_v<remove_cvref_t<U>, in_place_t>` is false, `and` `is_same_v<remove_cvref_t<U>, optional>` is false, `conjunction_v<is_aggregate<T>, negation<is_same<T, remove_reference_t<U>>>>` is false.